



Status	Started	Completed	Duration	Grade
Finished	Saturday, 7 June 2025, 9:50 AM	Saturday, 7 June 2025, 10:05 AM	14 mins 56 secs	70.00 out of 100.00

Information

Flag question

CAOS Eksamen afvikles som en moodle quizz;

- Det kan være en god ide at læse opgaverne igennem først, inden du begynder besvarelsen, så du kan vurdere, hvor du evt. skal prioritere for at samle flest points.
- Vær omhyggelig ved multiple-choice spørgsmål.
- Bemærk at rækkefølgen af nogle underspørgsmål og svarmuligheder "shuffles"
- Svarene kan frit ændres **indtil** den endelige indsendelse "finish attempt" inden eksamen afsluttes.
- Husk både af indsende besvarelsen i moodle ("finish attempt") og aflevere "Dokument to hand in" i digital eksamen. Det er tiden i digital eksamen, der er afgørende.

Dette eksamenssæt er delt op i 7 dele, der i alt giver 100 points. I hver del er der et eller flere spørgsmål.

1. Repræsentation og manipulation af information (15 pts)
2. Assembly programmer (20 pts)
3. Processor realisering (10 pts)
4. Program optimering og instruktionsniveau parallelitet (15 pts)
5. Operativ systemer (10 pts)
6. Hukommelses organisering (15 pts)
7. Multithreading og concurrency (15 pts)

Information

Flag question

Betragt den (hypothetiske) CAOSv1 maskine som anvender **w=14-bit words** til at repræsentere heltal. Det er givet at maskinen anvender two's complement til at repræsentere tal med fortegn (signed). Betrakt nedenstående erklæringer i C.

```
unsigned int x1 = UINT_MIN;
unsigned int x2 = UINT_MAX; //UMax w
int y1 = INT_MIN; //TMin w
int y2 = INT_MAX; //TMax w
int y3 = 1 ;
```

Nr.	Udtryk	Værdi i decimal	kommentar
1	x1	0	$B2U_{14}(00\ 0000\ 0000\ 0000)=0$
2	x2	16383	$B2U_{14}(11\ 1111\ 1111\ 1111)=16383$
3	y1	-8192	$B2T_{14}(10\ 0000\ 0000\ 0000)=-8192$
4	y2	8191	$B2T_{14}(01\ 1111\ 1111\ 1111)=8191$
5	y2+y3	-8192	overløb i positiv retning $T_{max}+1==T_{min}$
6	y1-y3	8191	overløb i negativ retning $T_{min}-1==T_{max}$
7	y1+x1	8192	blandet, beregnes som unsigned!
8	x2+y3	0	blandet, beregnes som unsigned, overflow
9	(y1+y2==0)	0 false	1 negativt tal mere end positive
10	(-y1==y1)	1 true	(-TMin=TMin)

Question 1

Flag question

Correct

Mark 4.00 out of 4.00

Udfyld nedenstående tabel med de tilhørende binære- og decimale-værdier for de erklærede variable.

Svar

Nr.	Udtryk	Biner Representation (uden spaces/opdeling)	Værdi (decimal)
1	x1	00000000000000 ✓	0 ✓
2	x2	11111111111111 ✓	16383 ✓
3	y1	10000000000000 ✓	-8192 ✓
4	y2	01111111111111 ✓	8191 ✓

Question 2

Flag question

Correct

Mark 6.00 out of 6.00

Udfyld nedenstående tabel for hvert af de angivne udtryk:

I delopgave 9 og 10 skal du bruge 0 for falsk og 1 for sand.

Svar

Nr	Udtryk	Beregnes som?	Overflow?	Værdi (decimal)
5	$y2+y3$	Signed ✓	Overflow i positiv retning ✓	-8192 ✓
6	$y1-y3$	Signed ✓	Overflow i negativ retning ✓	8191 ✓
7	$y1-x1$	Unsigned ✓	Ingen overflow ✓	8192 ✓
8	$x2+y3$	Unsigned ✓	Overflow i positiv retning ✓	0 ✓
9	$(y1+y2==0)$	Signed ✓	Ingen overflow ✓	0 ✓
10	$(-y1==y1)$	Signed ✓	Overflow i negativ retning ✓	1 ✓

Signed integers are represented using two's complement. Remark the asymmetrical numerical range: one more negative number than positive.

Nr.	Udtryk	Værdi i decimal	kommentar
1	$x1$	0	$B2U_{14}(00\ 0000\ 0000\ 0000)=0$
2	$x2$	16383	$B2U_{14}(11\ 1111\ 1111\ 1111)=16383$
3	$y1$	-8192	$B2T_{14}(10\ 0000\ 0000\ 0000)=-8192$
4	$y2$	8191	$B2T_{14}(01\ 1111\ 1111\ 1111)=8191$
5	$y2+y3$	-8192	overløb i positiv retning $T_{max}+1==T_{min}$
6	$y1-y3$	8191	overløb i negativ retning $T_{min}-1==T_{max}$
7	$y1+x1$	8192	blandet, beregnes som unsigned!
8	$x2+y3$	0	blandet, beregnes som unsigned, overflow
9	$(y1+y2==0)$	0 false	1 negativt tal mere end positive
10	$(-y1==y1)$	1 true	$(-T_{Min}=T_{Min})$

Question 3

Flag question

Correct

Mark 5.00 out of 5.00

Lad $x = 200$ decimal og $y = 0xBB$ hexadecimal, begge 8-bit (unsigned) ord. Udfyld tabellen med resultatet af nedenstående udtryk (Giv svar i hex, uden "0x" prefix!).

Bitwise operationer

Nr. Udtryk	Svar 0x...
1. $x y$	FB
2. $x \wedge \sim y$	8C
3. $x \gg 2$ (logisk)	32
4. $x \gg 2$ (aritmetisk)	F2
5. $x \ll 4$	80

```

x =      1100 1000
y =      1011 1011
x | y = 1111 1011 = FB = 251

x = 1100 1000  x = 1100 1000
y = 1011 1011 ~y = 0100 0100
x ^ ~y =          = 1000 1100 = 8C = 140

```

```

x = 1100 1000
x <<4 = 1000 0000 = 80 = 128

```

```

y = 1011 1011
y <<4 = 1011 0000 = B0 = 176

x = 1100 1000
x >>2 (logisk) = 0011 0010 = 32 = 50

x = 1100 1000
x >>2 (aritmetisk) = 1111 0010 = F2 = 242

```


Question 4

Flag question

Correct

Mark 5.00 out of 5.00

Hvilke af nedenstående udsagn er **SANDE**?

Select one or more:

- ☐ 1. **RET** instruktionen skubber returadressen på stakken.
- ☒ 2. Instruktionen **movl** flytter 4 bytes. ✓
- ☐ 3. Lad x være et heltal. Udtrykket $3*x$ kan i assembler beregnes som $x \ll 3$ (vha. instruktionen **shl**).
- ☒ 4. **%rdi** skal gemmes af den kaldende procedure (caller saved). ✓
- ☐ 5. **%rsi** skal gemmes af den kaldte procedure (callee saved).

Your answer is correct.

The correct answers are: **Instruktionen movl flytter 4 bytes, %rdi skal gemmes af den kaldende procedure (caller saved).**

Information

Flag question

Figuren viser et assembler program for funktionen "det" og et stykke hukommelse med et antal heltallige værdier.

Listing 2: Disassembleret oversat C program.

```

1 det :
2 0x4005b0 <+0>: movq    0x8(%rsi),%rax
3 0x4005b4 <+4>: imul    (%rdi),%rax
4 0x4005b8 <+8>: movq    (%rsi),%rdx
5 0x4005bb <+11>: imul    0x8(%rdi),%rdx
6 0x4005c0 <+16>: subq    %rdx,%rax
7 0x4005c3 <+19>: js      0x4005c7 <det+23>
8 0x4005c5 <+21>: retq
9 0x4005c7 <+23>: negq    %rax
10 0x4005ca <+26>: jmp     0x4005c5 <det+21>

```

Hukommelsen:	
Adresse	Værdi
0x601030	10
0x601038	8
0x601040	10
0x601048	9

Question 5

Flag question

Correct

Mark 3.00 out of 3.00

Angiv C-funktionens retur- og parameter-**typer**:

Type på retur værdi:

long



Type på første parameter:

long*



Type på anden parameter:

long*



Your answer is correct.

Du kan se, at de arbejde på longs, da det anvender 64-bit register navne (eller q-suffix på instruktionerne). Du kan se at det er en pointer, hvis det bruges som adresse i hukommelsen – husk, at når registeret står i parentes i en move instruktion betyder det at vi udlæser fra (eller skrive til) den adresse, der er angivet i registeret.

The correct answer is: **Type på retur værdi: → long, Type på første parameter: → long*, Type på anden parameter: → long***

Question 6

Flag question

Correct

Mark 2.00 out of 2.00

Koden ovenfor anvender en kendt kontrol struktur, hvilken?

- ☐ a. procedurekald
- ☐ b. while løkke
- ☐ c. for-løkke
- ☐ d. switch flervejs selektion
- ☐ e. do-while løkke
- ☒ f. if-then selektion

Your answer is correct.

Der er 'een forgrening i linie 7: et betinget hop js (jump-zero), som hopper til then delen (linie 9); ellers return. Dermed if-then (uden else, eller med tom else).

The correct answer is: **if-then selektion**

Question 7

Flag question

Correct

Mark 10.00 out of 10.00

Lav et beregningsspor (execution trace) af funktionen ved at udfylde nedenstående tabel givet at den kaldes med `det(0x601040,0x601030)`. Dvs. angiv effekten på processorens tilstanden ved at hånd-eksekvere instruktionerne én efter én. I linie n skal du angive den tilstand, der gælder inden instruktionen udføres; dens resultat (ændring af register-værdier) skal dermed stå i linien under: n+1. Hvis en værdi er ukendt, eller ikke kan udledes, skal du markere det med 'UK' for ukendt.

Listing 2: Disassembleret oversat C program.

```

1 det :
2 0x4005b0 <+0>: movq    0x8(%rsi),%rax
3 0x4005b4 <+4>: imul    (%rdi),%rax
4 0x4005b8 <+8>: movq    (%rsi),%rdx
5 0x4005bb <+11>: imul    0x8(%rdi),%rdx
6 0x4005c0 <+16>: subq    %rdx,%rax
7 0x4005c3 <+19>: js      0x4005c7 <det+23>
8 0x4005c5 <+21>: retq
9 0x4005c7 <+23>: negq    %rax
10 0x4005ca <+26>: jmp     0x4005c5 <det+21>

```

Hukommelsen:	
Adresse	Værdi
0x601030	10
0x601038	8
0x601040	10
0x601048	9

Beregningsspor

Skridt	PC (%rip) : Instruktion	%rdi	%rsi	%rdx	%rax	SF (signflag)
1	0x4005b0: movq 0x8(%rsi),%rax ✓	601040 ✓	601030 ✓	UK ✓	UK ✓	UK ✓
2	0x4005b4: imul (%rdi),%rax ✓	601040 ✓	601030 ✓	UK ✓	8 ✓	UK ✓
3	0x4005b8: movq (%rsi),%rdx ✓	601040 ✓	601030 ✓	UK ✓	80 ✓	0 ✓
4	0x4005bb: imul 0x8(%rdi),%rdx ✓	601040 ✓	601030 ✓	10 ✓	80 ✓	0 ✓
5	0x4005c0: subq %rdx,%rax ✓	601040 ✓	601030 ✓	90 ✓	80 ✓	0 ✓
6	0x4005c3: js 0x4005c7 ✓	601040 ✓	601030 ✓	90 ✓	-10 ✓	1 ✓
7	0x4005c7: negq %rax ✓	601040 ✓	601030 ✓	90 ✓	-10 ✓	1 ✓
8	0x4005ca: jmp 0x4005c5 ✓	601040 ✓	601030 ✓	90 ✓	10 ✓	0 ✓
9	0x4005c5: retq ✓	601040 ✓	601030 ✓	90 ✓	10 ✓	0 ✓
10	UK ✓	601040 ✓	601030 ✓	90 ✓	10 ✓	0 ✓
11	UK ✓	UK ✓	UK ✓	UK ✓	UK ✓	UK ✓

Det er oplyst i opgavet af funktionen "det(0x601040,0x601030)" kaldes. Første instruktion i koden ligger i adresse 0x4005b0 (movq), som programtælleren ved kaldet sættes til at udpege. Første og anden parametre overføres pr konvention i hhv rdi og rsi, og opsættes af kalderen til værdierne hhv. 0x601040 og 0x601030. Registrerne rdx, rax, og SF indholder de værdier, som de nu en gang har på kaldstidspunktet, og er dermed ukendte. Dermed er starte tilstanden i første linie fastlagt.

Effekten af movq i første skridt er at indholdet i adresse "8+%rsi" (=0x601038) kopieres over i %rax, som derved antager værdien 8. Denne tilstand skrives i linien under, og er dermed start-tilstanden for næste instruktion, Etc!

Bemærk at aritmetiske operationer (potentielt) ændrer SF.

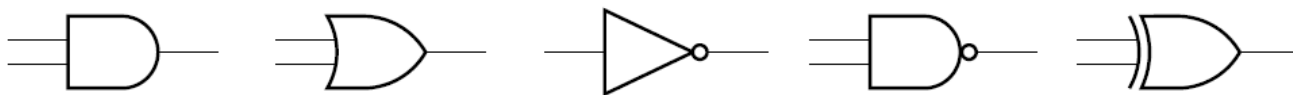
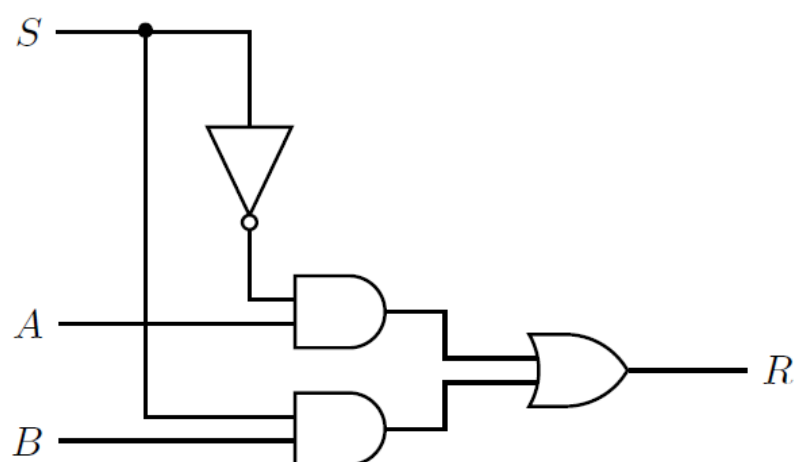
Beregningsspor

Skridt	PC (%rip) : Instruktion	%rdi	%rsi	%rdx	%rax	SF (signflag)
1	0x4005b0: movq 0x8(,%rdi,1) → %rdi	601040	601030	UK	UK	UK
2	0x4005b4: imul (%rdi,1) → %rdi	601040	601030	UK	8	UK
3	0x4005b8: movq (%rsi,1) → %rdi	601040	601030	UK	80	0
4	0x4005bb: imul 0x8(,%rdi,1) → %rdi	601040	601030	10	80	0
5	0x4005c0: subq %rdx, %rdi	601040	601030	90	80	0
6	0x4005c3: js 0x4005c7	601040	601030	90	-10	1
7	0x4005c7: negq %rax	601040	601030	90	-10	1
8	0x4005ca: jmp 0x4005c5	601040	601030	90	10	0
9	0x4005c5: retq	601040	601030	90	10	0
10	UK	601040	601030	90	10	0
11	UK	UK	UK	UK	UK	UK

Question 8

Flag question

Correct Mark 5.00 out of 5.00

Figur 1: *And, Or, Not, Nand og Xor* gates.

Figur 2: Et digitalt logisk kredsløb.

Figur 1 viser nogle logiske gates. Figur 2 anvender disse til opbygning af et digitalt logisk kredsløb.

Udfyld denne sandhedstabel for kredsløbet i Figur 2:

Sandhedstabel

A	B	S	R
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Question 9

Flag question

Correct

Mark 5.00 out of 5.00

I denne opgave antager vi processor arkitekturen SEQ Y86-64, som beskrevet i lærebogen.

Spor effekten af udførelsen af den konkrete instruktion ved at udfylde nedenstående tabel med specifikke konkrete **hexadecimale værdier** for instruktionen **push %rsi**. Det er givet, at **%rsp** har værdien **0x348**, **%rsi** har værdien **0x3**, og at program tælleren PC har værdien **0x77774402**. Alle værdier er underforstået HEX, udlad derfor 0x prefix.

Stage	pushq %rsi
Fetch	$\text{icode:ifun} \leftarrow M_1[\dots] = \dots : \dots$ $\text{rA:rB} \leftarrow M_1[\dots] = \dots : \dots$ $\text{valP} \leftarrow \dots = \dots$
Decode	$\text{valA} \leftarrow R[\dots] = \dots$ $\text{valB} \leftarrow R[\%rsp] = \dots$
Execute	$\text{valE} \leftarrow \text{valB} + (-8) = \dots$
Memory	$M_8[\dots] \leftarrow \dots$
Write back	$R[\%rsp] \leftarrow \dots$
PC update	$\text{PC} \leftarrow \dots$

Eksekvering (udlad 0x prefix)

Trin	push %rsi
Fetch	$\text{icode:ifun} \leftarrow M_1[77774402] = \text{a} : 0$ $\text{rA:rB} \leftarrow M_1[77774403] = 6 : \text{f}$ $\text{valP} \leftarrow 77774404$
Decode	$\text{valA} 3 \leftarrow R[6]$ $\text{valB} 348 \leftarrow R[4]$
Execute	$\text{valE} 340 \leftarrow \text{valB} + (-8)$
Memory	$M_8[340] \leftarrow 3$
Writeback	$R[4] \leftarrow 340$
PC update	$\text{PC} \leftarrow 77774404$

Læs fig 4.20 og Aside p 428 i bogen, og tilhørende tekst. Bemærk den anvendte notation $R[\]$, $M[\]$.

Stage	pushq rA
Fetch	$\text{icode:ifun} \leftarrow M_1[\text{PC}]$ $\text{rA:rB} \leftarrow M_1[\text{PC} + 1]$ $\text{valP} \leftarrow \text{PC} + 2$
Decode	$\text{valA} \leftarrow R[\text{rA}]$ $\text{valB} \leftarrow R[\%rsp]$
Execute	$\text{valE} \leftarrow \text{valB} + (-8)$
Memory	$M_8[\text{valE}] \leftarrow \text{valA}$
Write back	$R[\%rsp] \leftarrow \text{valE}$
PC update	$\text{PC} \leftarrow \text{valP}$

Figure 4.20 Computations in sequential pushq and popq. These instructions push a

Eksekvering (udlad 0x prefix)

Trin	push %rsi
Fetch	$\text{icode:ifun} \leftarrow M_1[77774402] = \alpha : 0$ $\text{rA:rB} \leftarrow M_1[77774403] = 6 : f$ $\text{valP} \leftarrow 77774404$
Decode	$\text{valA} 3 \leftarrow R[6]$ $\text{valB} 348 \leftarrow R[4]$
Execute	$\text{valE} 340 \leftarrow \text{valB} + (-8)$
Memory	$M_8[340] \leftarrow 3$
Writeback	$R[4] \leftarrow 340$
PC update	$\text{PC} \leftarrow 77774404$

Information

[Flag question](#)

Nedenfor er angivet en lille C-funktion **calc**, der foretager en beregning over elementerne i et array med N vektorer (repræsenteret som en struct `vec_t`). Funktionen leverer resultatet via resultat pointeren i parameter listen.

```
1  typedef struct vec {
2      long a;
3      long b;
4  } vec_t;
5  #define N 100000
6  vec_t v[N];
7
8  void calc(vec_t *v, const int vecLen, long* resultSum){
9      int i;
10     *resultSum=0;
11     for(i=0;i<vecLen;i++){
12         *resultSum+=v[i].a*v[i].a + v[i].b*v[i].b ;
13     }
14 }
15 int main(){
16     long total;
17     calc(v,N,&total);
18 }
```


Question 10

Flag question

Not answered

Marked out of 10.00

Unfolding

I et forsøg på at optimere denne funktion ønsker vi at lave en 3 gange udfoldning af løkken med 3 opsamlingsvariable (**3x3 loop unrolling**). Angiv den nødvendige kode hertil i funktionen **calc3x3**:

Hint: Hvis du er usikker på den eksakte syntaks, kan du stadig angive løsningen som pseudokode, evt. forklarende tekst.

Listing 6: Erklæringer.

```
1 long calcOptim3x3(vec_t *v, int vecLen, long*resultSum){
2
3     long res0=0;
4     long res1=0;
5     long res2=0;
6     const int limit=vecLen-2;
7     int i;
8     /*resultSum=0;
9     for(i=0;i<limit;i+=3){
10         res0+= v[i].a * v[i].a + v[i].b * v[i].b ;
11         res1+= v[i+1].a * v[i+1].a + v[i+1].b * v[i+1].b ;
12         res2+= v[i+2].a * v[i+2].a + v[i+2].b * v[i+2].b ;
13     }
14
15     for(;i<vecLen;i++){
16         res0+=v[i].a * v[i].a + v[i].b * v[i].b ;
17     }
18
19     res0=res0+res1+res2;
20     *resultSum=res0;
21 }
```

Husk:

Erklæring 3 akkumulator variable,

3 gentagelser af beregningerne i løkkens krop

Inkrementering af løkke-variabel med 3;

justering af slut-index for ikke at gå out of bounds

Behandling af resterende elementer inden retur

Question 11

Flag question

Not answered

Marked out of 5.00

Optimeringsbegrænsninger

Eksperimenter på en testmaskine viser at en program-transformation, som anvender en enkelt lokal akkumulator variabel (fx. 2×1 unrolling), giver en betragtelig hastighedsgevinst, men at en 3x3 udfoldning ikke giver nogen betydelig gevinst derudover.

Forklar kort med egne ord, hvorfor førstnævnte gevinst nåes, og en sandsynlig forklaring på at yderligere gevinst ikke opnåes.

Hint: Tænk i termer af "optimization blockers", og mulighed/begrænsning for parallel beregning.

1) Ved at optælle i en lokal variabel, fjerner vi **hukommelses-aliaseringen** mellem resultat pointeren og vektoren. Dette er vigtigt a) for at akkumuleringen kan ske i et register uden dyr mellemliggende læsning/skrivning til hukommelsen, og b) ligeledes nødvendigt for at multiplikationerne i en udfoldning ($v[i]$ og $v[i + 1]$) kan ske parallelt, da aliaseringen kan forårsage at resultatet skrives til $v[i + 1]$.

Optimeringen her er sikker og korrekt da der ikke kan opstå aliasering da resultatet er sat op i main til at blive leveret i en lokal variabel, som er adskilt fra arrayet!!

2) I 3x3 udfoldningen er der lagt op til 6 parallelle multiplikationer og 6+1 summer. Men det er sandsynligt, at processor-kernen, som afvikler programmet har færre multiplikations-enheder, og at vi har nået dens **produktivitetstærskel**. Endvidere bliver der også pres på load-enhederne til indlæsning af 6 longs fra hukommelsen pr iteration.

(Eksempel arkitekturen i lærebogen har faktisk kun een enhed til heltalsmultiplikation:fuldt pipelinet med latency 3)

Question 12

Flag question

Correct

Mark 5.00 out of 5.00

Vælg de egenskaber, der kendetegner multi-programmering

- ☒ a. Med multi-programmering kan processoren afvikle flere programmer tilsyneladende samtidigt. ✔ Ja, det er næsten definitionen på multiprogrammering
- ☒ b. Multi-programmering tillader multiple brugere. ✔ Ja, flere samtidige processer gør at flere brugere kan have en eller flere processer kørende samtidigt, så det ser ud som om de har hver deres (mindre kraftige) processor.
- ☐ c. Multi-programmeringer kun en fordel når processoren har flere kerner (til parallel beregning).
- ☒ d. Multi-programmering gør det muligt at opnå større udnyttelsesgrad af processoren. ✔ Ja, når en process afventer i/o kan en anden process overtage processoren.
- ☒ e. Multi-programmering gør det muligt at opnå større udnyttelsesgrad af ydre enheder. ✔ Ja, En process kan sætte en disk i gang med at indlæse; samtidigt med kan en anden process forberede næste læsning, eller aktivere en helt anden ydre enhed.
- ☐ f. Multi-programmering gør det muligt at at operativ systemet kan programmeres af flere udviklere.

Your answer is correct.

The correct answers are: Multi-programmering gør det muligt at opnå større udnyttelsesgrad af processoren.,

Multi-programmering tillader multiple brugere.,

Med multi-programmering kan processoren afvikle flere programmer tilsyneladende samtidigt.,

Multi-programmering gør det muligt at opnå større udnyttelsesgrad af ydre enheder.

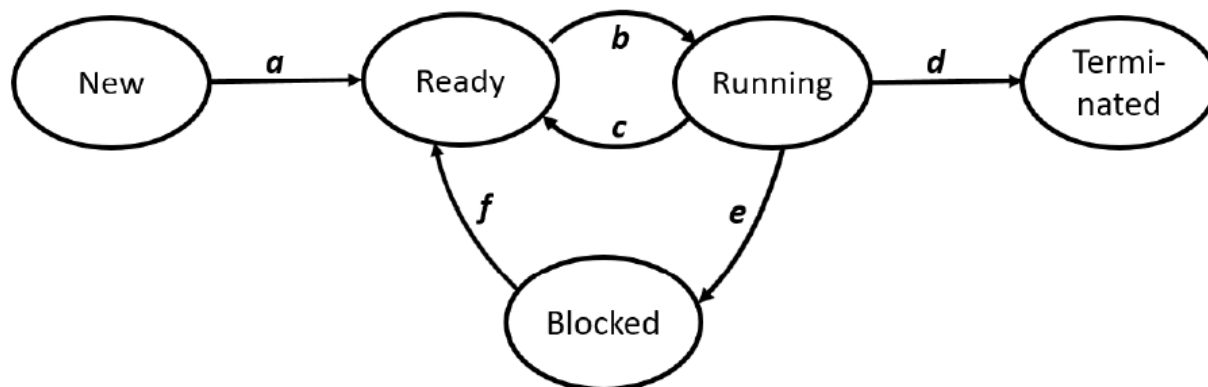
Question 13

Flag question

Correct

Mark 5.00 out of 5.00

Betragt trådtilstandsmodellen i nedenstående figur.



Angiv med bogstaverne a – f den tilstandsovergang, der svarer til nedenstående situationer:

Tråden kalder wait på en semafor med værdien 0

 ✓

Pagefault indtræffer ved adgang til en allokeret side, men som ikke er resident i RAM.

 ✓

Tråden kalder yield

 ✓

Tråden kalder en længerevarende synkron I/O operation

 ✓

En kørende tråd afbrydes af et timer interrupt, som angiver, at dens timeslice er udløbet

 ✓

Your answer is correct.

Angiv med bogstaverne a – f den tilstandsovergang, der svarer til nedenstående situationer:

Tråden kalder yield

Pagefault indtræffer ved adgang til en allokeret side, men som ikke er resident i RAM.

Tråden kalder wait på en semafor med værdien 0

En kørende tråd afbrydes af et timer interrupt, som angiver, at dens timeslice er udløbet

Tråden kalder en længerevarende synkron I/O operation

The correct answer is: **Tråden kalder wait på en semafor med værdien 0 → e,**

Pagefault indtræffer ved adgang til en allokeret side, men som ikke er resident i RAM. → e,Tråden kalder yield → c,

Tråden kalder en længerevarende synkron I/O operation → e,

En kørende tråd afbrydes af et timer interrupt, som angiver, at dens timeslice er udløbet → c

Information

Flag question

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Question 14

Flag question

Correct Mark 5.00 out of 5.00

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Vis hvordan den virtuelle adresse **0x150A** oversættes til fysisk adresse. Brug minus symbolet "-" til at angive at en værdi ikke kan genereres.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	0	0	0	1	0	1	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Adresse oversættelse

Parameter	Værdi
VPN (0x...):	A8 ✓
TLB Indeks (0x...):	0 ✓
TLB-Tag (0x...):	2A ✓
TLB-Hit (j/n):	n ✓
Page fault (j/n):	n ✓
PPN (0x...):	06 ✓

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	0	0	1	0	1	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Sidestørrelse på 32 bytes betyder at VPO er 5 bits. Så bits 0-4 bruges til VPO, bits 5-13 VPN.

TLB-I kræver 2 bits da TLB cache har 4 sets. TLB-I findes i bits 5+6. TLB-T findes i bits 7-13.

Bits i virtuel adresse

VPN							VPO						
13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	0	0	0	1	0	1	0
TLB-Tag							TLB-Index						

Bits i fysisk adresse

PPN							VPO				
11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	0	0	1	0	1	0

Question 15

Flag question

Correct

Mark 5.00 out of 5.00

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Vis hvordan den virtuelle adresse **0x1590** oversættes til fysisk adresse. Brug minus symbolet "-" til at angive at en værdi ikke kan genereres.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	0	0	1	0	0	0	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Adresse oversættelse

Parameter	Værdi
VPN (0x...):	AC ✓
TLB Indeks (0x...):	0 ✓
TLB-Tag (0x...):	2B ✓
TLB-Hit (j/n):	j ✓
Page fault (j/n):	n ✓
PPN (0x...):	4A ✓

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	1	0	1	0	0	0	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Sidestørrelse på 32 bytes betyder at VPO er 5 bits. Så bits 0-4 bruges til VPO, bits 5-13 VPN.

TLB-I kræver 2 bits da TLB cache har 4 sets. TLB-I findes i bits 5+6. TLB-T findes i bits 7-13.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	0	0	1	0	0	0	0

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	1	0	1	0	0	0	0

Question 16

Flag question

Correct Mark 5.00 out of 5.00

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Vis hvordan den virtuelle adresse **0x15EA** oversættes til fysisk adresse. Brug minus symbolet "-" til at angive at en værdi ikke kan genereres.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	1	1	0	1	0	1	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Adresse oversættelse

Parameter	Værdi
VPN (0x...):	AF ✓
TLB Indeks (0x...):	3 ✓
TLB-Tag (0x...):	2B ✓
TLB-Hit (j/n):	n ✓
Page fault (j/n):	j ✓
PPN (0x...):	- ✓

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	-	-	-	-	-	-	-	-
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Sidestørrelse på 32 bytes betyder at VPO er 5 bits. Så bits 0-4 bruges til VPO, bits 5-13 VPN.

TLB-I kræver 2 bits da TLB cache har 4 sets. TLB-I findes i bits 5+6. TLB-T findes i bits 7-13.

Når der er sidefejl, kan der ikke generes nogen fysisk adresse (og dermed heller intet PPN). Dert er dermed en fejl at forsøge at generere fysisk adresse.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	1	1	0	1	0	1	0

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	-	-	-	-	-	-	-	-

Information

[Flag question](#)

Nedenfor er skitseret en lille Pthreads program, med 3 tråde, der (potentielt) parallelt skal beregne summen af elementerne i et (hypotetisk lille) array.

Hver tråd skal bruge den samlede sum som et del-resultat i en videre beregning, som ikke er en del af opgaven. En tråd gemmer sin delsum i et andet 3-element array på index svarende til sit **id**, der bliver overført til tråden som parameter.

For at virke korrekt, skal det sikres at hver tråd afventer de 2 andres del-summer i linie 22 (markeret som kommentar i koden) inden de beregner den samlede sum. Altså, inden en tråd fortsætter i linie 24, skal den sikre at de 2 øvrige har afsluttet beregningen op til linie 19.

Vis vha. Pthreads **mutexes og condition** variable hvordan du kan opnå den krævede synkronisering.

Hint: programmet er listet som verbatim, så det er nemt at copy-paste ind i din virtuelle maskine til videre-udvikling.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include "common.h"
#include "common_threads.h"

const int N=3;
long arr[9]={1,2,3, 4,5,6, 7,8,9};
long sum[3]={0,0,0};

//Mutex og condition erklæringer

void *mythread(void *arg) {
    long id=(long)arg;
    printf("Traad %ld: start\n", id);
    long total=0;
    for(int i=0;i<N;i++)
        total+= arr[id*N+i];
    sum[id]=total;
    printf("Traad %ld: foer: %ld\n", id, total);

    //22 Traaden skal vente her paa de andre traade

    total=0;
    for(int i=0;i<N;i++) total+=sum[i];
    printf("Traad %ld: efter: %ld\n", id, total);
    //fortsat beregning; ikke en del af opgaven.
    return NULL;
}

int main(int argc, char *argv[]) {
    pthread_t t1, t2,t3,t4;
    Pthread_create(&t1, NULL, mythread, (void*)0);
    Pthread_create(&t2, NULL, mythread, (void*)1);
    Pthread_create(&t3, NULL, mythread, (void*)2);
    Pthread_join(t1, NULL);
    Pthread_join(t2, NULL);
    Pthread_join(t3, NULL);
    return 0;
}
```

Question 17

Flag question

Not answered

Marked out of 3.00

Del 1/3. Opstil de nødvendige erklæringer og initialisering af mutex og condition variable, samt eventuelle globale delte variable.

Listing 7: Erklæringer.

```
1 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
2 pthread_cond_t waiters = PTHREAD_COND_INITIALIZER;  
3 int noArrived=0;
```

Question 18

Flag question

Not answered

Marked out of 7.00

Del 2/3. Opstil koden til synkronisering, som kan indsættes omkring linie 22.

Listing 8: Synkroniseringskode.

```
1
2 Pthread_mutex_lock(&mutex);
3 noArrived++;
4
5 if(noArrived==N) {
6     Pthread_cond_broadcast(&waiters);
7 } else {
8     while(noArrived<N) {
9         Pthread_cond_wait(&waiters,&mutex);
10    }
11 }
12 Pthread_mutex_unlock(&mutex);
```

Question 19

[Flag question](#)

Not answered

Marked out of 5.00

Del 3/3. Beskriv kort virkemåden af din løsning, eller løsningsidé.

Ideen er at lade en global variable optælle antallet af ankomne tråde. Sålænge antallet ikke har nået 3, skal trådene blokeres vha. `cond_wait`. Den sidst ankomne vækker alle øvrige vha `cond_broadcast`. Optælling af antallet af ankomne tråde skal ske udeleligt, og derfor beskyttes med en lås. Endvidere skal brug af wait og (bør) signal/broadcast på en condition variable ske mens låsen holdes. Vi husker at vække ventende så der ikke bliver baglås. Kaldet til broadcast sker kun når antallet af fremmødte er N, så unødvendig vækning/kald til `condbroadcast` undgås.

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 10:09 AM	Completed Saturday, 7 June 2025, 10:16 AM
Duration 6 mins 45 secs	Marks 11.00/11.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 2.00 out of 2.00

En bit kan repræsentere 2 mulige værdier, benævnt 0 og 1 (false og true).

Med 2 bits kan vi lave 4 forskellige kombinationer, og dermed 4 forskellige værdier:

bit 1	bit 0
0	0
0	1
1	0
1	1

En byte er en samling af 8 bits. Hvor mange forskellige værdier kan repræsenteres ved en byte?



Et ord (word) er en samling af et antal bits i en enhed, der repræsenterer et typeløst tal.

Hvor mange forskellige værdier kan repræsenteres af et ord, som er 2 bytes (w=16 bits) langt?



Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Binær til decimal: Hvilken decimal-værdi har det binære tal 01110110?

Answer:



Højre mod venstre:

$$0111\ 0110 = 0 \cdot 2^0 + 1 \cdot 2^1 + 1 \cdot 2^2 + 0 \cdot 2^3 + 1 \cdot 2^4 + 1 \cdot 2^5 + 2^6 + 0 \cdot 2^7 = \\ 0 + 2 + 4 + 0 + 16 + 32 + 64 + 0 = 118$$

The correct answer is: 118

Question 3

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hex til decimal: Hvilken decimal (10-tals system) værdi har hexadecimal tallet 0xA5B7?

Answer:



$$0xA5B7 = 7 \cdot 16^0 + 11 \cdot 16^1 + 5 \cdot 16^2 + 10 \cdot 16^3 = 7 + 176 + 1280 + 40960 = \\ 42423$$

The correct answer is: 42423

Question 4

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Binær til hex: Hvilken hexa-decimal-værdi har det binære tal 01110110? 0x...

Answer:



Opdel i grupper af 4 cifre fra højre mod venstre, og konverter hver gruppe til hex

0110 = 6

0111 = 7

Opdel i grupper af 4 cifre fra højre mod venstre, og konverter hver gruppe til hex

0110 = 6

0111 = 7

The correct answer is: 76

Question 5

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Binær til hex: Hvilken hexa-decimal-værdi har det binære tal 110110? 0x...

Answer:



Opdel i grupper af 4 cifre fra højre mod venstre, og konverter hver gruppe til hex

0110 = 6

11 = 3

The correct answer is: 36

Question 6

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Angiv tallet 0xA5B7 på binær form (2-tals system).
(angiv svar uden whitespaces)

Answer:



Højre mod venstre

7=0111

B= 1011

5= 0101

A= 1010

The correct answer is: 1010010110110111

Question 7

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Angiv tallet 0x9FF som et 16-bit binært ord.
(brug ingen whitespaces i svar)

Answer:



De **mest-betydende** bits skal 0 stilles, da de ikke skal bidrage til tallet.

The correct answer is: 0000100111111111

Question 8

[Flag question](#)

Correct

Mark 3.00 out of 3.00

Lad $a=0x28$.

Hvilken hexadecimal værdi har $a-0x8$? (angiv svar uden 0x prefix)



Hvilken hexadecimal værdi har $a+0x8$? (angiv svar uden 0x prefix)



Hvilken hexadecimal værdi har $a+0xf$? (angiv svar uden 0x prefix)

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 10:20 AM	Completed Saturday, 7 June 2025, 10:37 AM
Duration 16 mins 31 secs	Marks 8.00/8.00	Grade 10.00 out of 10.00 (100%)

Question 1

Flag question

Correct

Mark 7.00 out of 7.00

Figuren nedenfor viser en model af hukommelsen med noget indhold (alle tal i hex):

Bytes

				48	65	6A	20	6D	65	64	20	6A	65	73	21	0	...	
Addr.	00000000	00000001	00000002	00000003	00000004	00000005	00000006	00000007	00000008	00000009	0000000A							ffffff

Antag at hukommelsen er 2^{32} bytes stor.

- Hvad er den mindste adresse i hex? 0x ✓
- Hvad er den højeste adresse i hex? 0x ✓
- Hvad er den højeste adresse som decimal: ✓

Antag at strengen "Hej med jer!" er kodet op i [ascii](#) og er gemt i hukommelsen med start i adresse 0x4. Da hver karakter fylder een byte giver det følgende sekvens af ascii værdier (i hex, inklusiv C's "0"-terminering af strenge): 48,65,6A, 20, 6d, 65, 64, 20, 6A, 65,73,21,0.

- Hvad er værdien af byten som er gemt i adresse 0xA? 0x... ✓
- I hvilken adresse ligger karakteren "!" gemt? 0x... ✓
- Hvad er decimal værdien af den byte som er gemt i adresse 0x8? ✓
- Hvilken karakter er gemt i adresse 0x4+0xB? ✓

Question 2

Flag question

Correct

Mark 1.00 out of 1.00

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.

In a computer using LITTLE endian, a 16-bit word starts at address 0x514 ? What hex-value is stored there? 0x

f456



In a computer using BIG endian, what is the address containing the least significant byte 0x7B2 0x

517



In a computer using LITTLE endian, what is the address containing the least significant byte 0x7B2 0x

510



In a computer using LITTLE endian, what is the address containing the most significant byte 0x1A2 0x

517



In a computer using BIG endians, what is the address containing the most significant byte 0x1A2 0x

510



In a computer using BIG endian, what is the value of the byte stored in address 0x514 ? 0x..

bc



In a computer using LITTLE endian, what is the value of the byte stored in address 0x514 ? 0x

56



517

d5

f4

510

bc

f456

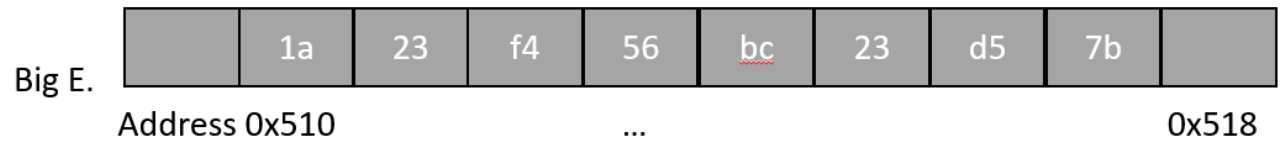
56f4

56

5a0

Your answer is correct.

The 64-bit word 0x1A23F456BC23D57B is stored in memory starting at address 0x510.
Consider the difference between little end and big end byte order:

[Finish review](#)



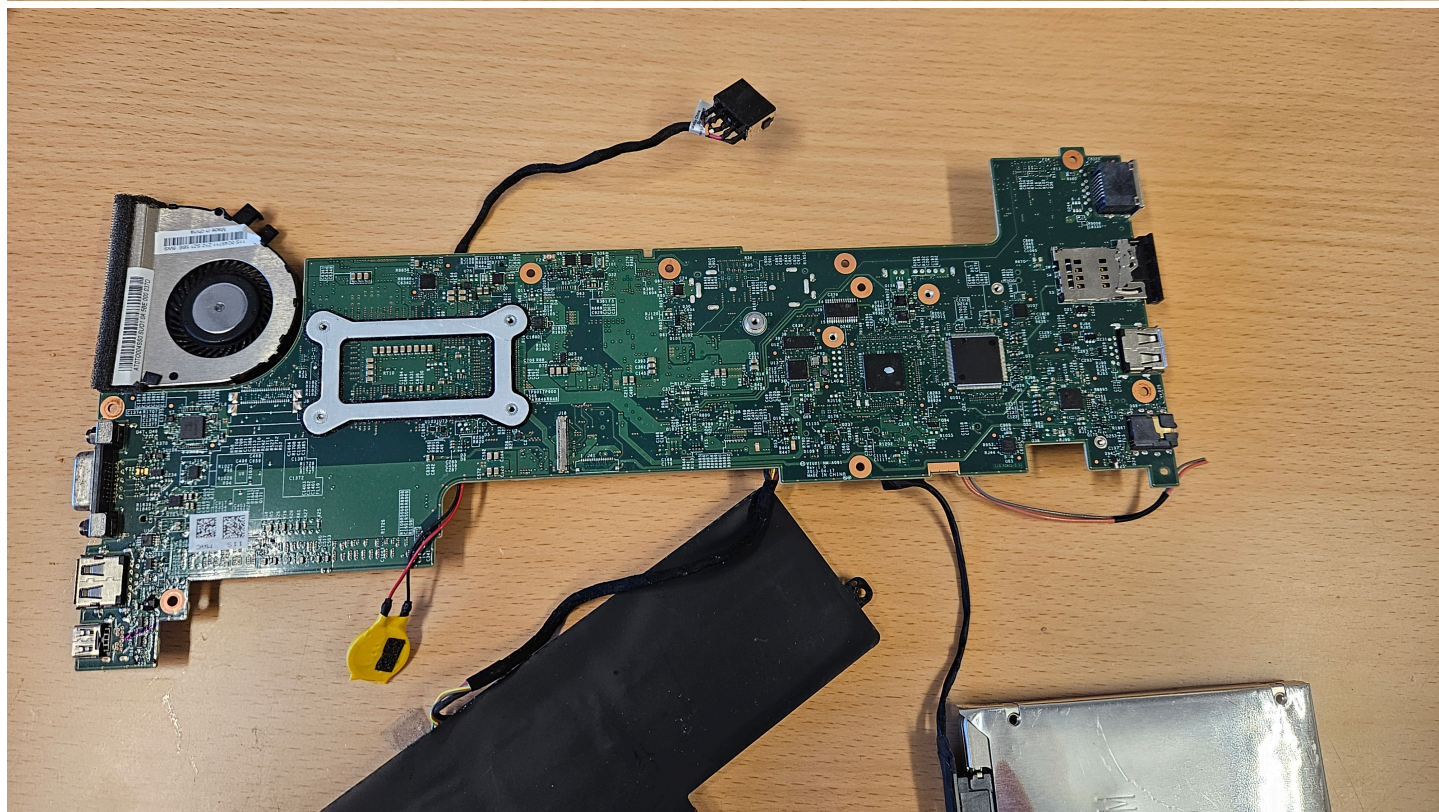
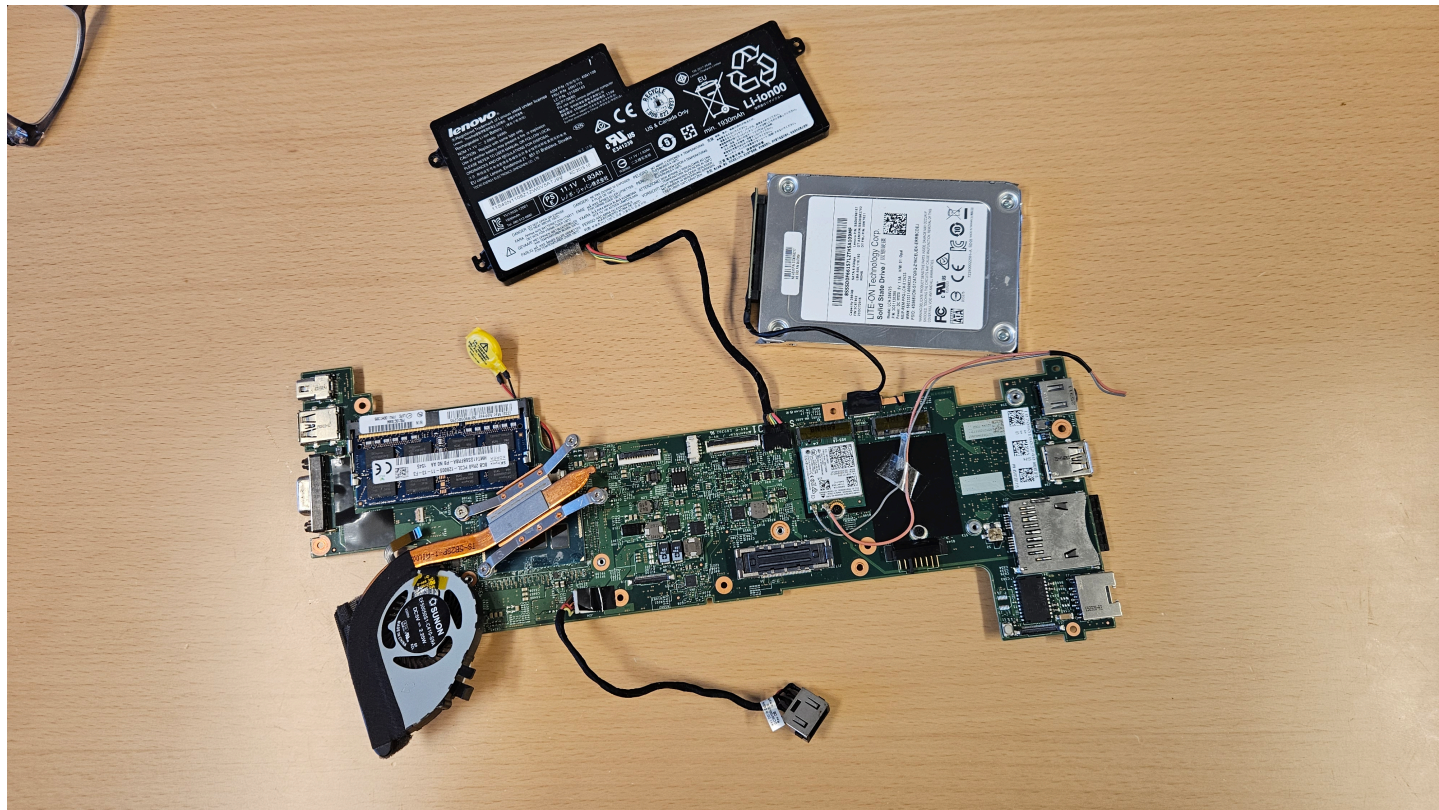
Status	Started	Completed	Duration
Finished	Saturday, 7 June 2025, 10:39 AM	Saturday, 7 June 2025, 10:46 AM	6 mins 31 secs
Grade	Not yet graded		

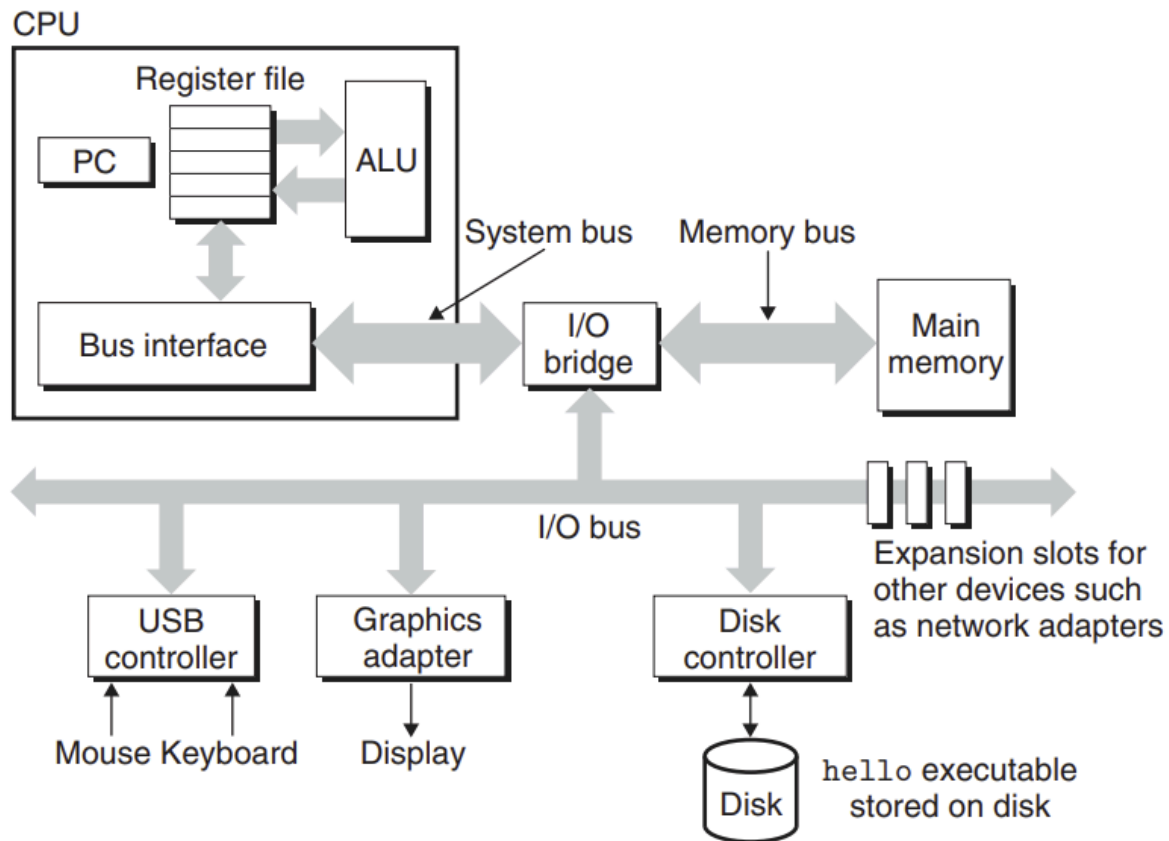
Information

Flag question

På nedenstående billeder har jeg "skræillet" en (defekt/uddateret) bærbar Lenovo PC, dvs. fjernet kabinet, tastatur, skærm og diverse, så bundkortet (2-sidet printplade) med de væsentligste komponenter fremgår.

Ligeledes er vist et diagram fra en lærebog over hardware organiseringen i en computer, altså en "abstraktion" over de elektroniske komponenter i maskinen.





Question 1

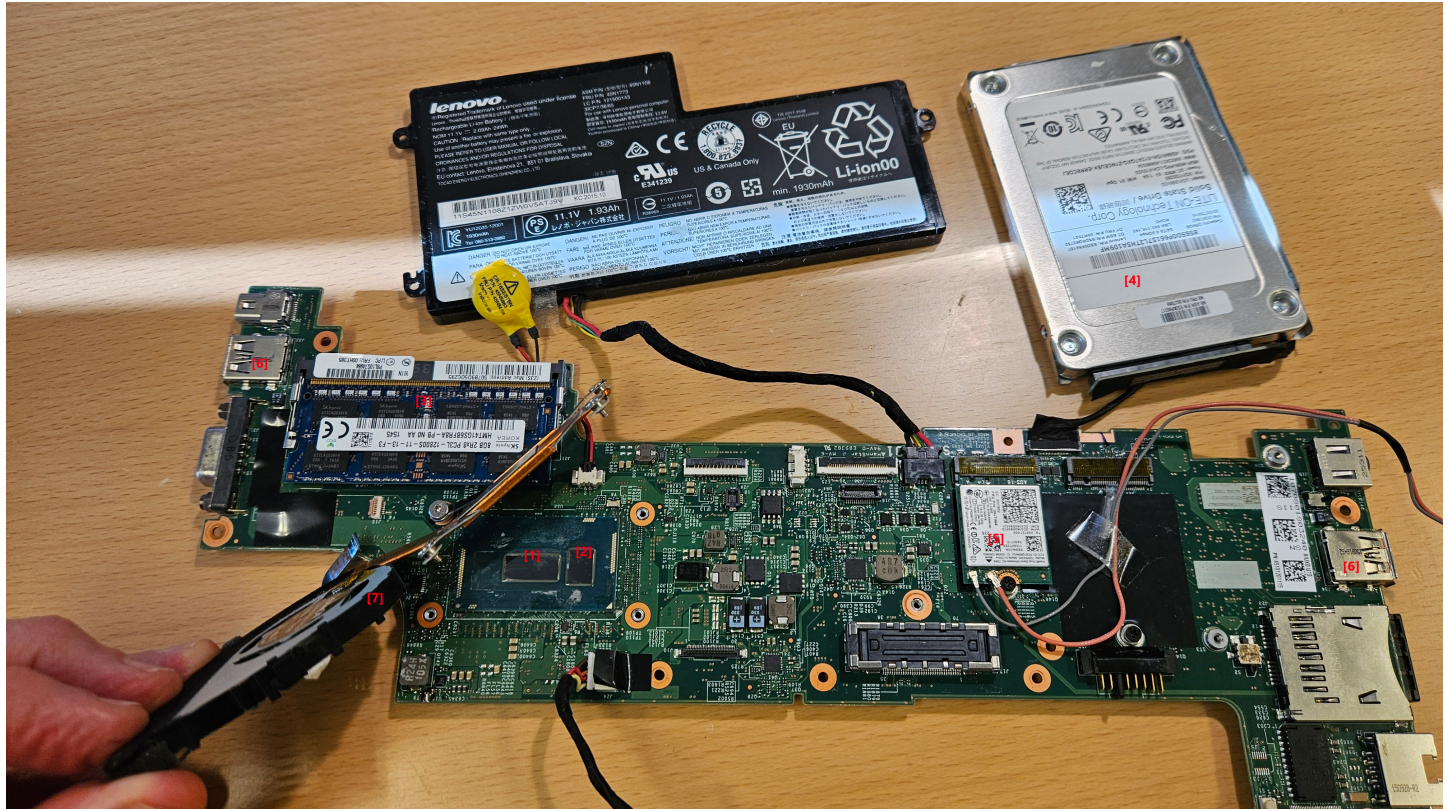
Flag question

Correct

Mark 10.00 out of 10.00

Betragt numrene på nedenstående billede af bundkortet af en skrallet bærbar PC, og det abstrakte diagram over computerens organisering, som det typisk findes i lærebøger (Fx CSPP3ed Fig 1.4).

Prøv at matche numrene på billedet med de væsentligste system komponenter.



CPU / processor (specifikt Intel i5)

[1]



Sekundær lager / Disk

[4]



Primær Hukommelsen / main-memory / arbejdshukommelsen / RAM

[3]

I/O- og System-Bridge alias PCH ([Platform Controller HUB](#))

[2]



USB port

[6]



Wifi Adapter

[5]



Blæser og køle-element

[7]



Your answer is correct.

The correct answer is: CPU / processor (specifikt Intel i5) → [1], Sekundær lager / Disk → [4],

Primær Hukommelsen / main-memory / arbejdshukommelsen / RAM → [3], I/O- og System-Bridge alias PCH ([Platform Controller HUB](#))

→ [2], USB port → [6], Wifi Adapter → [5], Blæser og køle-element → [7]

Question 2

[Flag question](#)

Complete

Marked out of 3.00

Hvad er formålet med kølelementet og hvorfor er det nødvendigt?

En CPU er opbygget af et milliard-stort antal transistorer, der bygges sammen til gigantiske logiske kredsløb. En transistor kan skifte mellem en "åben" og "lukket" tilstand vha. en styrespænding. Når den skifter tilstand, bruges en lille mængde energi (strøm), som afsættes som varme-effekt omkring transistoren. Når mange transistorer skifter meget hurtigt (1 Giga-hertz ~ 1 milliard skifte per sekund) afsættes meget effekt på et meget lille fysisk område (ca 1 cm², se billede). Denne varme skal transporteres bort, ellers brænder CPU'en bogstaveligt talt af.

(CPU'en i en mellem kraftig bærbar computer forbruger ca 60 watt - svarende til en kraftig gammeldags glødepære, som du kan forbrænde fingrene slemt på. En kraftig stationær kan kræve flere hundrede watt.)

Man bliver altså dobbelt straffet: energi til at drive processoren og tilsvarende mængde til at køle den. Store datacentre kræver derfor enorme mængder energi!!

En CPU er opbygget af et milliard-stort antal transistorer, der bygges sammen til gigantiske logiske kredsløb. En transistor kan skifte mellem en "åben" og "lukket" tilstand vha. en styrespænding. Når den skifter tilstand, bruges en lille mængde energi (strøm), som afsættes som varme-effekt omkring transistoren. Når mange transistorer skifter meget hurtigt (1 Giga-hertz ~ 1 milliard skifte per sekund) afsættes meget effekt på et meget lille fysisk område (ca 1 cm², se billede). Denne varme skal transporteres bort, ellers brænder CPU'en bogstaveligt talt af.

(CPU'en i en mellem kraftig bærbar computer forbruger ca 60 watt - svarende til en kraftig gammeldags glødepære, som du kan forbrænde fingrene slemt på. En kraftig stationær kan kræve flere hundrede watt.)

Man bliver altså dobbelt straffet: energi til at drive processoren og tilsvarende mængde til at køle den. Store datacentre kræver derfor enorme mængder energi!!

Question 3

Flag question

Complete

Marked out of 3.00

Hvor på printpladen findes mon "system-" "memory-" og "I/O-busserne"?

Busserne findes på flerlagsprintpladen som elektriske forbindelser, typisk et større antal parallelle forbindelser.

I denne bærbare computer er "system bussen" integreret i samme integrerede pakke som CPU og PCH, mens "memory bus" og "I/O bus"

(

De er tydeligere hvis du analyserer meget simpel hardware, eller meget gammel PC:

Forbindelse til IO-bus på (meget gammel) 16-bit PC (Ethernet interface kort). Der ses 16 adresse ben, 8 data ben, nogle kontrol

Busserne findes på flerlagsprintpladen som elektriske forbindelser, typisk et større antal parallelle forbindelser.

I denne bærbare computer er "system bussen" integreret i samme integrerede pakke som CPU og PCH, mens "memory bus" og "I/O bus" findes på flerlags-printpladen hhv. gående mod RAM modulet og mod højre til fx wifi-og USB controller/portene. De er så højt integrerede og små, at de nærmest er usynlige.

(

De er tydeligere hvis du analyserer meget simpel hardware, eller meget gammel PC:

Forbindelse til IO-bus på (meget gammel) 16-bit PC ([Ethernet interface kort](#)). Der ses 16 adresse ben, 8 data ben, nogle kontrol ben (fx kontrollerer om der læses eller skrives til kortet, clock, mv.).



)

Finish review



Status Finished	Started Saturday, 7 June 2025, 10:52 AM	Completed Saturday, 7 June 2025, 10:53 AM
Duration 33 secs	Grade 6.00 out of 6.00 (100%)	

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvad er information?

Select one:

- ☐ a. Bits
- ☒ b. Bits og kontekst ✓
- ☐ c. Bytes
- ☐ d. Kontekst og metadata

Your answer is correct.

The correct answer is: Bits og kontekst

Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Det hexadecimale talsystem har basetallet

Select one:

☐ a. 1☐ b. 2☐ c. 8☐ d. 10☒ e. 16 ✓

Your answer is correct.

The correct answer is: 16

Question 3

[Flag question](#)

Correct

Mark 1.00 out of 1.00

$0x16 + 0xf =$

Select one:

☐ 0x26

☐ 0x1f

☒ 0x25 ✓

☐ 0x106

Your answer is correct.

The correct answer is: 0x25

Question 4

[Flag question](#)

Correct

Mark 1.00 out of 1.00

42 i decimal er i hexadecimal:

Select one:

- ☐ a. 0x42
- ☐ b. 0x24
- ☒ c. 0x2a ✓
- ☐ d. 0x32

Your answer is correct.

The correct answer is: 0x2a

Question 5

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvilket talsystem regner computere i?

Select one:

- ☒ a. Binært ✓
- ☐ b. Oktalt
- ☐ c. Decimalt
- ☐ d. Hexadecimalt

Your answer is correct.

The correct answer is: Binært

Question 6

[Flag question](#)

Correct

Mark 1.00 out of 1.00

De fleste kan tælle til 10 ved at bruge deres fingre. I har nu lært om det binære talsystem.

Hvor meget kan der tælles til på fingrene, ved at bruge det binære talsystem og lade hver finger repræsentere et ciffer?

Answer:



The correct answer is: 1023

Question 7

[Flag question](#)

Complete

Not graded

Har du fået den virtuelle maskine op at køre?

Select one:

☒ True

☐ False

The correct answer is 'True'.

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 10:54 AM	Completed Saturday, 7 June 2025, 11:00 AM
Duration 6 mins 41 secs	Marks 7.00/7.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 3.00 out of 3.00

Betragt 8-bit ordet $w=10101010$.

1. Hvilken værdi har w fortolket som unsigned (decimal)? ✓
2. Hvilken værdi har w fortolket som signed two's complement (decimal)? ✓
3. Hvilken two's complement bit-repræsentation skal w have for at gemme decimal værdien -8? ✓

Vægte af bits: $128+64+32+16+8+4+2+1$

Unsigned: $1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 128+32+8+2 = 170$

Two's complement: $-1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = -128+32+8+2 = -86$

$-8 = -128+64+32+16+8$ svarende til bitstrengen 1111000

Question 2

[Flag question](#)

Correct

Mark 4.00 out of 4.00

Consider a (hypothetical) CAOSv1 machine that uses **6-bit words** to represent integer numbers. Give answers as decimal numbers.

1. What is the smallest **unsigned** integer that the machine can represent as a single word?



2. What is the largest **unsigned** integer that the machine can represent as a single word?



3. What is the smallest **signed** integer that the machine can represent as a single word?



4. What is the largest **signed** integer that the machine can represent as a single word?



Signed integers are represented using two's complement. Remark the asymmetrical numerical range: one more negative number than positive.

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:02 AM	Completed Saturday, 7 June 2025, 11:16 AM
Duration 14 mins 4 secs	Marks 7.00/7.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 7.00 out of 7.00

Betragt (den hypotetiske) CAOSv1 maskine som anvender **6 bits** til repræsentation af heltal (integers). Det er endvidere givet at den anvender **two's complement** til repræsentation af signed integers og two's complement addition på signed integers.

A C-program has the following declarations:

```
unsigned int x1 = UINT_MIN ;
unsigned int x2 = UINT_MAX; //UMax_w
int y1 = INT_MIN ; //TMin_w
int y2 = INT_MAX; //TMax_w
int y3 = 6;
```

Beregn værdien af nedenstående C-udtryk. Giv resultater som decimal værdier (10 talssystem):

Integer udtryk:

1. $x2+1$ ✓
2. $x2+y3$ ✓
3. $x1-y3$ ✓
4. $y1-1$ ✓
5. $y1-y3$ ✓
6. $y2+y3$ ✓
7. $x2+y1$ ✓

Consider text book equation 2.11 for unsigned overflow and 2.13 for signed overflow.

1. Unsigned! Overflow i positiv retning, $(63_U+1_U)=0 = (63+1)\%2^w$

2. Unsigned! Overflow i positiv retning, $63_U+(1+5)_U = 0+5 = (63+6)\%2^w$

3. Unsigned! Overflow i negativ retning, $(0-1-5)U = 63-5 = 58$ el. $(0U + (-6)U) \% 2^w = (0 + B2U_6(1111010)) \% 2^w = 58 \% 2^w$

4. Signed! Overflow i negativ retning, $-32-1 = 31 = -33 + 2^w$

5. Signed! Overflow i negativ retning, $-32-1-5 = 31-5=26 = -38 + 2^w$

6. Signed! Overflow i positiv retning, $31+1+5 = -32+5=27 = 36 - 2^w$

7. Unsigned! $63U + (-32)U = 63+B2U(10000) = 63+32$: Overflow i positiv retning, $95 \% 2^w = 31$

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:19 AM	Completed Saturday, 7 June 2025, 11:19 AM
Duration 4 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

How many iterations will the loop in the following program execute?

```
#include<stdio.h>
int main () {
    for (double i = 10; i != 0; i = i - 0.1) {
        printf("%.15f\n",i);
    }
}
```

- ☐ a. 100
- ☐ b. 0
- ☐ c. 101
- ☐ d. 99
- ☒ e. A lot! ✓

Your answer is correct.

Ikke alle reelle tal har en eksakt repræsentation i floats eller doubles!

Program output:

10.0000000000000000
9.9000000000000000
9.8000000000000001
9.7000000000000001
9.6000000000000001
9.5000000000000002
9.4000000000000002
9.3000000000000002
9.2000000000000003
9.1000000000000003
9.0000000000000004
8.9000000000000004
8.8000000000000004
8.7000000000000005
8.6000000000000005
8.5000000000000005
8.4000000000000006
8.3000000000000006
8.2000000000000006
8.1000000000000007
8.0000000000000007
7.9000000000000007
7.8000000000000008
7.7000000000000008
7.6000000000000009
7.5000000000000009
7.4000000000000009
7.3000000000000010
7.2000000000000010
7.1000000000000010
7.0000000000000011
6.9000000000000011
6.8000000000000011
6.7000000000000012
6.6000000000000012
6.5000000000000012
6.4000000000000013
6.3000000000000013
6.2000000000000014
6.1000000000000014
6.0000000000000014
5.9000000000000015
5.8000000000000015
5.7000000000000015
5.6000000000000016
5.5000000000000016
5.4000000000000016
5.3000000000000017
5.2000000000000017
5.1000000000000017
5.0000000000000018
4.9000000000000018
4.8000000000000018
4.7000000000000019
4.6000000000000019

4.500000000000020
4.400000000000020
4.300000000000020
4.200000000000021
4.100000000000021
4.000000000000021
3.900000000000021
3.800000000000021
3.700000000000021
3.600000000000021
3.500000000000021
3.400000000000021
3.300000000000021
3.200000000000021
3.100000000000021
3.000000000000020
2.900000000000020
2.800000000000020
2.700000000000020
2.600000000000020
2.500000000000020
2.400000000000020
2.300000000000020
2.200000000000020
2.100000000000020
2.000000000000020
1.900000000000019
1.800000000000019
1.700000000000019
1.600000000000019
1.500000000000019
1.400000000000019
1.300000000000019
1.200000000000019
1.100000000000019
1.000000000000019
0.900000000000019
0.800000000000019
0.700000000000019
0.600000000000019
0.500000000000019
0.400000000000019
0.300000000000019
0.200000000000019
0.100000000000019
0.000000000000019
-0.099999999999981
-0.199999999999981
-0.299999999999981
-0.399999999999981
-0.499999999999981
-0.599999999999981
-0.699999999999981
-0.799999999999981

...

The correct answer is: **A lot!**

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:20 AM	Completed Saturday, 7 June 2025, 11:24 AM
Duration 3 mins 48 secs	Marks 6.00/6.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 6.00 out of 6.00

Beregn resultatet af følgende udtryk af 8 bit ord med bitvise operationer.

Q1

Udtryk Værdi

x	11001000
---	----------

y	10101010
---	----------

x y	11101010	✓
-------	----------	---

Q2

Udtryk Værdi

x	11001000
---	----------

y	10101010
---	----------

x ^ ~y	10011101	✓
--------	----------	---

Q3

Udtryk Værdi

y	10101010
---	----------

y << 4	10100000	✓
--------	----------	---

Q4

Udtryk

Værdi

x	11001000
---	----------

x >>3 (logisk)	00011001	✓
----------------	----------	---

Q5**Udtryk****Værdi****Udtryk****Værdi**

x	11001000
---	----------

x >> 3 (aritmetisk)	1111001	✓
---------------------	---------	---

Q6**Udtryk****Værdi**

x	01001000
---	----------

x >> 3 (aritmetisk)	00001001	✓
---------------------	----------	---

x = 1100 1000

```

y =      1010 1010
x | y = 1110 1010 = 0xEA = 234

x = 1100 1000  x = 1100 1000
y = 1010 1010 ~y = 0101 0101
x ^ ~y =          = 1001 1101 = 0x9D = 157

```

y = 1010 1010

```

y <<4 = 1010 0000 = 0xA0 = 160

x = 1100 1000
x >>3 (logisk) = 0001 1001 = 0x19 = 25

x = 1100 1000
x >>3 (aritmetisk) = 1111 1001 = 0xF9 = 249

```

```

x = 0100 1000
x >>3 (aritmetisk) = 0000 1001 = 0x09 = 9

```

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:25 AM	Completed Saturday, 7 June 2025, 11:27 AM
Duration 2 mins 37 secs	Marks 4.00/4.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

En 32-bit integer kan repræsentere intervallet:

Select one:

- ☒ a. $[-2^{31}, 2^{31}-1]$ ✓
- ☐ b. $[-2^{32}, 2^{31}-1]$
- ☐ c. $[0, 2^{32}]$
- ☐ d. $[-1, 32768]$

Your answer is correct.

The correct answer is: $[-2^{31}, 2^{31}-1]$

Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

En signed int repræsenteres vha.

Select one:

- ☐ a. Sign-Magnitude encoding
- ☒ b. Two's complement ✓
- ☐ c. One's complement

Your answer is correct.

The correct answer is: Two's complement

Question 3

[Flag question](#)

Correct

Mark 1.00 out of 1.00

For to (signed) integers, x og y , gælder det at $x+y$ kan være

Select one:

- ☒ a. Negativt eller positivt ✓
- ☐ b. Positivt
- ☐ c. Negativt

Your answer is correct.

The correct answer is: Negativt eller positivt

Question 4

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Checker dette program korrekt for integer overflow?

```
unsigned int foo(unsigned int x, unsigned int y) {  
    unsigned int s;  
    s = x + y;  
    if (s < x)  
        error();  
    return s;  
}
```

Select one:

- ☐ a. Nej.
- ☒ b. Ja. ✓

Your answer is correct.

The correct answer is: Ja.

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:29 AM	Completed Saturday, 7 June 2025, 11:29 AM
Duration 27 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvilken af følgende x86-64 instruktioner beregner korrekt $\%rax = 9 * \%rdi$?

- ☒ a. `leaq (%rdi,%rdi,8), %rax`  Korrekt: $\%rdi + 8 * \%rdi$ gemmes i `%rax`
- ☐ b. `leaq (,%rdi,9), %rax`
- ☐ c. `movq (,%rdi,9), %rax`
- ☐ d. `movq (%rdi,%rdi,8), %rax`

Your answer is correct.

Lea beregner effektiv adresse.

Move flytter data til/fra hukommelsen angivet ved den beregnede adresse. Dermed kan vi udelukke de 2 muligheder med `moveq`!

Adresse beregning: se den generelle format: $D(Rb, Ri, S)$

Betydning: $Reg[Rb] + S * Reg[Ri] + D$

Her skal skaleringsfaktoren være 1,2,4,8; 9 er dermed IKKE tilladt.

`leaq (%rdi, rdi,8)` beregner således adressen $(\%rdi + \%rdi * 8)$ ialt $9 * \%rdi$!

The correct answer is: **`leaq (%rdi,%rdi,8), %rax`**

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:30 AM	Completed Saturday, 7 June 2025, 11:33 AM
Duration 2 mins 33 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Match disse x86-64 registre med deres brug i et typisk program.

%rax	General purpose	✓
%rcx	General purpose	✓
%rbp	General purpose	✓
%rip	Instruction pointer	✓
%rdx	General purpose	✓
%rsp	Stack pointer	✓

Your answer is correct.

(%rbp bruges ofte af compiler til at udpege start på stack-frame)

The correct answer is: %rax → General purpose, %rcx → General purpose, %rbp → General purpose, %rip → Instruction pointer, %rdx → General purpose, %rsp → Stack pointer

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:39 AM	Completed Saturday, 7 June 2025, 11:40 AM
Duration 59 secs	Marks 3.00/3.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Lad

```
%rdx=0xf000 og %rcx= 0x0100
```

Betrakt nedenstående instruktionen:

```
movq 0xa(%rdx,%rcx,8), %rax
```

Fra hvilken effektiv adresse udlæses værdien som skal gemmes i %rax? 0x...

Answer:



Format: D(Rb,Ri,S)

Betydning: $\text{Reg}[\text{Rb}] + \text{S} * \text{Reg}[\text{Ri}] + \text{D}$

Altså $0xf000 + 8 * 0x100 + a = f80a$

The correct answer is: f80a

Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Lad

```
%rdx=0xf000 og %rcx= 0x0100
```

Betragt nedenstående instruktionen:

```
leaq 0xa(%rdx,%rcx,8), %rax
```

Hvilken værdien gemmes i %rax? 0x...

Answer:



Format: D(Rb,Ri,S)

Betydning: $\text{Reg}[\text{Rb}] + \text{S} * \text{Reg}[\text{Ri}] + \text{D}$

Altså $0xf000 + 8 * 0x100 + a = f80a$

The correct answer is: f80a

Question 3

Flag question

Correct

Mark 1.00 out of 1.00

Betragt nedenstående tilstand for registre og hukommelsen før og efter en sekvens af assembler-instruktioner er kørt. Instruktionen afvikles i rækkefølgen spørgsmålene kommer i.

Hvilken sekvens af instruktioner resulterer i efter-tilstanden?

Før

Registre		Hukommelsen	
%rdi	0x520	123	Adresse 0x520
%rsi	0x500		0x518
%rax			0x510
%rdx			0x508
		456	0x500

Efter

Registre		Hukommelsen	
%rdi	0x500	123	Adresse 0x520
%rsi	0x500		0x518
%rax	0x123		0x510
%rdx		123	0x508
		456	0x500

1. Hvilken instruktion vil give %rax værdien 123?

movq (%rdi), %rax



2. Hvilken instruktion vil skrive 0x500 til %rdi registeret?

movq %rsi, %rdi



3. Hvilken instruktion vil skrive 0x123 til hukommelsesadresse 0x508?

movq %rax,8(%rdi)



Your answer is correct.

The correct answer is:

1. Hvilken instruktion vil give %rax værdien 123? → movq (%rdi), %rax,

2. Hvilken instruktion vil skrive 0x500 til %rdi registeret? → movq %rsi, %rdi,

3. Hvilken instruktion vil skrive 0x123 til hukommelsesadresse 0x508? → movq %rax,8(%rdi)



Status Finished	Started Saturday, 7 June 2025, 11:42 AM	Completed Saturday, 7 June 2025, 11:47 AM
Duration 4 mins 28 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Betrakt afviklingen af nedenstående assembler

```
imul    %rdi,%rdi
imul    %rsi,%rsi
add     %rsi,%rdi
lea     (%rdi,%rdi,1),%rax
```

Hvilket udtryk beregner det (resultat i %rax) når programmet startes med %rdi havende startværdi x, og %rsi havende startværdi y?

- ☐ a. $(2*x+2*y) + x+y$
- ☐ b. $2*(2*x+2*y)$
- ☐ c. $(2*x+2*y) + x+y+1$
- ☒ d. $2*(x*x+y*y)$ ✓
- ☐ e. $(x*x+y*y) + x+y+1$

 $rdi = rdi * rdi // = x * x$ $rsi = rsi * rsi // = y * y$ $rdi = rdi + rsi // x * x + y * y$ $rax = (rdi + rdi * 1) = 2 * rdi = 2 * (x * x + y * y)$

Your answer is correct.

The correct answer is: **$2*(x*x+y*y)$**

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 11:48 AM	Completed Saturday, 7 June 2025, 12:06 PM
Duration 18 mins 19 secs	Marks 50.00/50.00	Grade 10.00 out of 10.00 (100%)

Question 1

Flag question

Correct

Mark 48.00 out of 48.00

I det følgende er vist et fragment af et X86-64 assembly program, som gcc har oversat fra et lille C-program.

Lav et beregningsspor (execution trace) af funktionen ved at udfylde nedenstående tabel givet start tilstanden givet i første linie. Ved start har %rdi værdien 4. Du kan antage, at der ikke sker overflow.

Dvs. angiv effekten på processorens tilstanden ved at hånd-eksekvere instruktionerne én efter én. I linie n skal du angive den tilstand, der gælder inden instruktionen udføres; dens resultat (ændring af register-værdier) skal dermed stå i linien under: n+1. Hvis en værdi ikke kan udledes, bedes du angive dette med et "-". Stands når/hvis eksekvering når "ret" og lad efterfølgende være markeret med "-".

```
0000000000401136 <func>:
 401136: mov    %rdi,%rax
 401139: jmp    40113f <func+0x9>
 40113b: sub    $0x2,%rax
 40113f: cmp    $0x1,%rax
 401143: jg     40113b <func+0x5>
 401145: ret
```

Beregningsspor

	PC (%rip) 0x... : instruktionsnavn	%rax 0x...	SF	ZF
	401136: mov %rdi,%rax	-	-	-
1	401139: jmp 40113f	4	-	-
2	40113f: cmp \$0x1,%rax	4	-	-
3	401143: jg 40113b	4	0	0
4	40113b: sub \$0x2,%rax	4	0	0
5	40113f: cmp \$0x1,%rax	2	0	0
6	401143: jg 40113b	2	0	0
7	40113b: sub \$0x2,%rax	2	0	0
8	40113f: cmp \$0x1,%rax	0	0	1
9	401143: jg 40113b	0	1	0
10	401145: ret	0	1	0

10	701145: ret	✓	0	✓	1	✓	0	✓
----	-------------	---	---	---	---	---	---	---

	PC (%rip)0x... : instruktionsnavn		%rax 0x...		SF		ZF	
11	-	✓	-	✓	-	✓	-	✓
12	-	✓	-	✓	-	✓	-	✓

	PC (%rip)0x... : instruktionsnavn		%rax 0x...		SF		ZF	
	401136: mov %rdi,%rax		-		-		-	
1	401139: jmp 40113f ↕		4 ↕		- ↕		- ↕	
2	40113f: cmp \$0x1,%rax ↕		4 ↕		- ↕		- ↕	
3	401143: jg 40113b ↕		4 ↕		0 ↕		0 ↕	
4	40113b: sub \$0x2,%rax ↕		4 ↕		0 ↕		0 ↕	
5	40113f: cmp \$0x1,%rax ↕		2 ↕		0 ↕		0 ↕	
6	401143: jg 40113b ↕		2 ↕		0 ↕		0 ↕	
7	40113b: sub \$0x2,%rax ↕		2 ↕		0 ↕		0 ↕	
8	40113f: cmp \$0x1,%rax ↕		0 ↕		0 ↕		1 ↕	
9	401143: jg 40113b ↕		0 ↕		1 ↕		0 ↕	
10	401145: ret ↕		0 ↕		1 ↕		0 ↕	
11	- ↕		- ↕		- ↕		- ↕	
12	- ↕		- ↕		- ↕		- ↕	

cmp: 4:1
jg 4>1

cmp: 2:1
jg 2>1

cmp: 0:1
jg 0>1

jg Greater (Signed): (fig 3.15)

$\sim(\text{SF} \wedge \text{OF}) \& \sim \text{ZF}$

Given OF=0:

$\sim(\text{SF} \wedge 0) \& \sim \text{ZF}$

Jump if:

SF=0 & ZF=0

Question 2

Flag question

Correct

Mark 1.00 out of 1.00

Nedenfor er vist et stykke assembler, som er oversat af gcc fra en C-funktion.

```
401136: mov %rdi,%rax
401139: jmp 40113f
40113b: sub $0x2,%rax
40113f: cmp $0x1,%rax
401143: jg 40113b
401145: ret
```

Fra hvilken af nedenstående C-funktioner stammer assembler koden:

C-funktioner

func0	func1	func2	func3
<pre>long func0(long x){ while(x>=2){ x=x-2; } return x; }</pre>	<pre>long func1(long x){ do{ x=x-2; } while(x>=2); return x; }</pre>	<pre>long func2(long x){ do{ x=x-2; } while(x>1); return x; }</pre>	<pre>long func3(long x){ x=x-2; if(x>1) return x; else x=1; return x; }</pre>

- ☐ a. func3
- ☐ b. func2
- ☒ c. func0 ✓
- ☐ d. func1

Your answer is correct.

Vi ser at der er et conditional jmp tilbage i koden: 401143: jg 40113b

Dermed en iteration

Vi genkender spring frem til evaluering af betingelse

```
401139: jmp 40113f
```

Vi genkender dermed "Jump-to-middle" oversættelse, og dermed en while

Endelig, negerer vi udtrykket $x > 1$ til $x \leq 2$

```
long t= %rdi
goto test;
12:
    t-=2
test:
    compare t,1
    if (t>1) goto 12.
```

Generel oversættelse af "While" #1

- "Jump-to-middle" oversættelse
- Bruges med -Og

While: Jump-to-middle

While version

```
while (Test)
    Body
```



Goto Version

```
goto test;
loop:
    Body
test:
    if (Test)
        goto loop;
done:
```

```
long t= x    #kalds-konvention
while(t>1)  #iteration, jump-to middle
    t-=2      #body
Return t    #kaldskonvention
```

The correct answer is: **func0**

Question 3

Flag question

Correct

Mark 1.00 out of 1.00

Koden nedenfor anvender en kendt kontrol struktur, hvilken?

Listing 2: Disassembleret oversat C program.

```
1 det :  
2 0x4005b0 <+0>: movq    0x8(%rsi),%rax  
3 0x4005b4 <+4>: imul    (%rdi),%rax  
4 0x4005b8 <+8>: movq    (%rsi),%rdx  
5 0x4005bb <+11>: imul    0x8(%rdi),%rdx  
6 0x4005c0 <+16>: subq    %rdx,%rax  
7 0x4005c3 <+19>: js      0x4005c7 <det+23>  
8 0x4005c5 <+21>: retq  
9 0x4005c7 <+23>: negq    %rax  
10 0x4005ca <+26>: jmp     0x4005c5 <det+21>
```

- ☒ a. if-then selektion ✓
- ☐ b. procedurekald
- ☐ c. while løkke
- ☐ d. for-løkke
- ☐ e. do-while løkke
- ☐ f. switch flervejs selektion

Your answer is correct.

Der er 'een forgrening i linie 7: et betinget hop js (jump-zero), som hopper til then delen (linie 9); ellers return. Dermed if-then (uden else, eller med tom else).

The correct answer is: **if-then selektion**



Status

Finished

Started

Saturday, 7 June 2025, 3:10 PM

Completed

Saturday, 7 June 2025, 3:19 PM

Duration

9 mins 10 secs

Marks

19.00/19.00

Grade

10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 18.00 out of 18.00

I det følgende er vist et fragment af et (uoptimeret) X86-64 assembly program, som gcc har oversat fra et lille C-program.

Lav et beregningsspor (execution trace) af funktionen ved at udfylde nedenstående tabel givet start tilstanden givet i første linie.

Dvs. angiv effekten på processorens tilstanden ved at hånd-eksekvere instruktionerne én efter én. I linie n skal du angive den tilstand, der gælder inden instruktionen udføres; dens resultat (ændring af register-værdier) skal dermed stå i linien under: n+1. Hvis en værdi ikke kan udledes, kan du angive dette med et "-"

Listing 1: Oversat C program

```

1 0x400517: movq %rdi,%rax
2 0x40051a: addq (%rsi),%rax
3 0x40051d: retq
4 0x40051e: subq $0x10,%rsp
5 0x400522: movl $0x14,0x8(%rsp)
6 0x40052b: leaq 0x8(%rsp),%rsi
7 0x400530: movl $0xa,%edi
8 0x400535: call 0x400517
9 0x40053a: addq $0x10,%rsp
10 0x40053e: retq

```

Beregningsspor

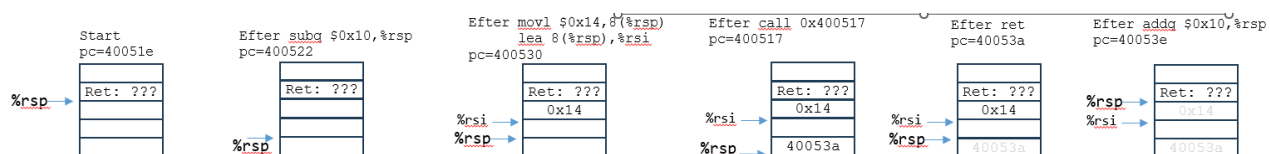
	PC (%rip)0x... : instruktionsnavn		%rsp 0x...	
1	0x40051e : subq \$0x10 ,%rsp		7fde28	
2	0x400522 : movl \$0x14, 0x8(%rsp)	✓	7fde18	✓
3	0x40052b : leaq 0x8(%rsp), %rsi	✓	7fde18	✓
4	0x400530 : movl \$0xa, %edi	✓	7fde18	✓
5	0x400535 : call 0x400517	✓	7fde18	✓
6	0x400517 : movq %rdi, %rax	✓	7fde10	✓
7	0x40051a : addq (%rsi), %rax	✓	7fde10	✓
8	0x40051d : retq	✓	7fde10	✓
9	0x40053a : addq \$0x10, %rsp	✓	7fde18	✓

✓	0x40053a : addq \$0x10, %rsp	✓	7fde18	✓
---	------------------------------	---	--------	---

	PC (%rip)0x... : instruktionsnavn		%rsp 0x...	
10	0x40053e : retq	✓	7fde28	✓

Beregningsspor

	PC (%rip)0x... : instruktionsnavn	%rsp 0x...
1	0x40051e : subq \$0x10, %rsp	7fde28
2	0x400522 : movl \$0x14, 0x8(%rsp) ⇅	7fde18 ⇅
3	0x40052b : leaq 0x8(%rsp), %rsi ⇅	7fde18 ⇅
4	0x400530 : movl \$0xa, %edi ⇅	7fde18 ⇅
5	0x400535 : call 0x400517 ⇅	7fde18 ⇅
6	0x400517 : movq %rdi, %rax ⇅	7fde10 ⇅
7	0x40051a : addq (%rsi), %rax ⇅	7fde10 ⇅
8	0x40051d : retq ⇅	7fde10 ⇅
9	0x40053a : addq \$0x10, %rsp ⇅	7fde18 ⇅
10	0x40053e : retq ⇅	7fde28 ⇅



Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

I det følgende er vist et fragment af et (uoptimeret) X86-64 assembly program, som gcc har oversat fra et lille C-program.

Listing 1: Oversat C program

```

1 0x400517:  movq %rdi,%rax
2 0x40051a:  addq (%rsi),%rax
3 0x40051d:  retq
4 0x40051e:  subq $0x10,%rsp
5 0x400522:  movl $0x14,0x8(%rsp)
6 0x40052b:  leaq 0x8(%rsp),%rsi
7 0x400530:  movl $0xa,%edi
8 0x400535:  call 0x400517
9 0x40053a:  addq $0x10,%rsp
10 0x40053e:  retq

```

Hvilken værdi (decimal tal) returnerer funktionen når programtælleren (%rip) fra start antager

0x40051e?



I det følgende er vist et fragment af et (uoptimeret) X86-64 assembly program, som gcc har oversat fra et lille C-program.

	Listing 1: Oversat C program	#f: (rdi, rsi)
1	0x400517: movq %rdi,%rax	tmp res (i rax)=rdi
2	0x40051a: addq (%rsi),%rax	res += *rsi
3	0x40051d: retq	return res (i rax)
4	0x40051e: subq \$0x10,%rsp	//alloc stack plads
5	0x400522: movl \$0x14,0x8(%rsp)	tmp1=20 (=0x14)
6	0x40052b: leaq 0x8(%rsp),%rsi	rsi=&tmp1
7	0x400530: movl \$0xa,%edi	tmp2 (i rdi)=10 (0xa)
8	0x400535: call 0x400517	call f (tmp2, &tmp1) # rdi,rsi
9	0x40053a: addq \$0x10,%rsp	//dealloc stack plads
10	0x40053e: retq	return %rax

```
long func1(long a, long *b) {return a+*b;}
```

```
long func() {long a=10; long b=20; return func1(a,&b);}
```



Status Finished	Started Saturday, 7 June 2025, 3:29 PM	Completed Saturday, 7 June 2025, 3:31 PM
Duration 2 mins 52 secs	Marks 5.00/5.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct Mark 5.00 out of 5.00

1. Instruktionen `movl` flytter 4 bytes. ✓
2. Lad x være et heltal. Udtrykket $3 * x$ kan i assembler beregnes som $x \ll 3$ (vha. instruktionen `shl`).
 ✓
3. `RET` instruktionen skubber returadressen på stakken. ✓
4. `%rdi` skal gemmes af den kaldende procedure (caller saved). ✓
5. `%rsi` skal gemmes af den kaldte procedure (callee saved). ✓

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 3:32 PM	Completed Saturday, 7 June 2025, 3:42 PM
Duration 9 mins 37 secs	Marks 4.00/4.00	Grade 10.00 out of 10.00 (100%)

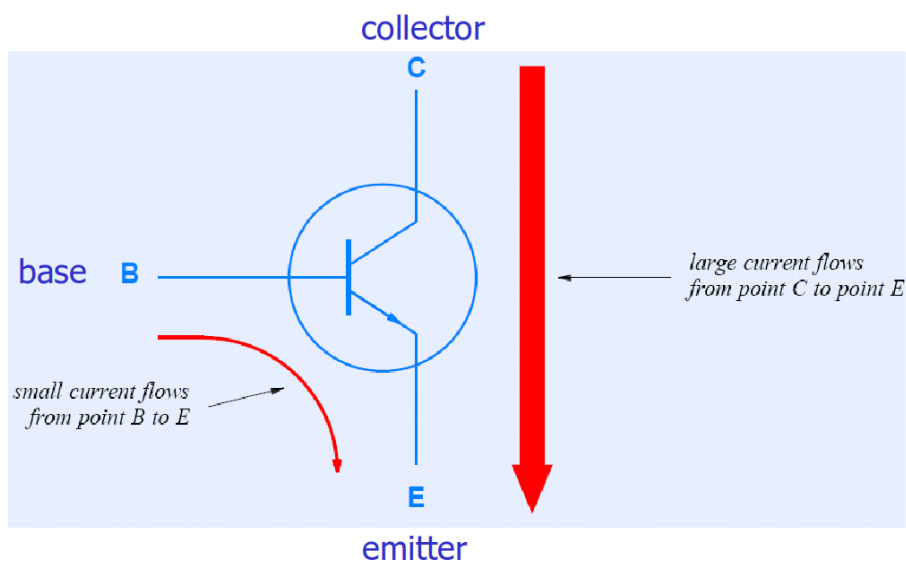
Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvad er en transistor?



Select one:

- ☐ a. En clock-cycle generator
- ☐ b. En elektronisk memory-enhed
- ☒ c. En elektronisk tænd-sluk knap ✓

Your answer is correct.

The correct answer is: En elektronisk tænd-sluk knap

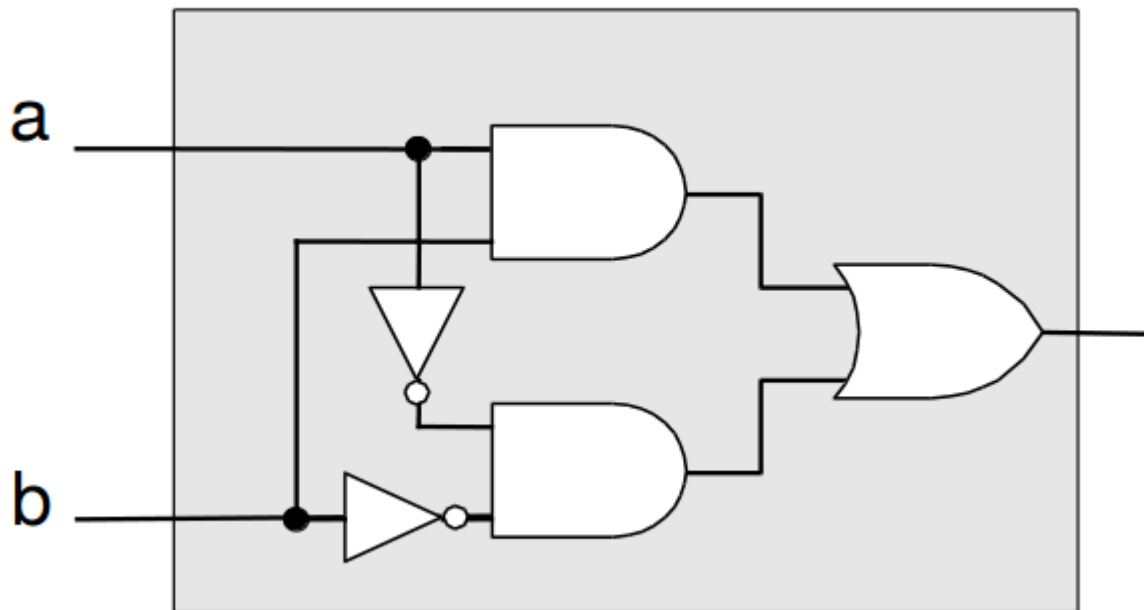
Question 2

Flag question

Correct

Mark 1.00 out of 1.00

Lav sandheds-tabellen for dette digitale kredsløb



$a=0, b=0$ ✓

$a=0, b=1$ ✓

$a=1, b=0$ ✓

$a=1, b=1$ ✓

Your answer is correct.

The correct answer is: $a=0, b=0 \rightarrow 1, a=0, b=1 \rightarrow 0, a=1, b=0 \rightarrow 0, a=1, b=1 \rightarrow 1$

Question 3

[Flag question](#)

Correct

Mark 1.00 out of 1.00

SEQ processoren har 6 stages. Match disse med deres funktionalitet:

Fetch	Læs instruktion fra memory	✓
Decode	Find værdier der skal beregnes på	✓
Memory	Foretag hukommelses-tilgang	✓
PC	Opdater program counter	✓
Writeback	Skriv værdier til registre	✓
Execute	Foretag ALU-beregning	✓

Your answer is correct.

The correct answer is: Fetch → Læs instruktion fra memory, Decode → Find værdier der skal beregnes på, Memory → Foretag hukommelses-tilgang, PC → Opdater program counter, Writeback → Skriv værdier til registre, Execute → Foretag ALU-beregning

Question 4

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvor mange clock cycles bruger SEQ processoren på at udføre én instruktion?

Select one:

- ☒ a. 1 ✓
- ☐ b. 2
- ☐ c. 3
- ☐ d. 4
- ☐ e. 5
- ☐ f. 6

Your answer is correct.

The correct answer is: 1

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 3:43 PM	Completed Saturday, 7 June 2025, 3:49 PM
Duration 5 mins 18 secs	Marks 15.00/15.00	Grade 10.00 out of 10.00 (100%)

Question 1

Flag question

Correct Mark 15.00 out of 15.00

I denne opgave antager vi processor arkitekturen SEQ Y86-64, som beskrevet i lærebogen. Spor effekten af udførelsen af den konkrete instruktion ved at udfylde nedenstående tabel med specifikke konkrete **hexadecimale værdier** for instruktionen **pushq %rsi**. Det er givet, at **%rsp** har værdien **0x348**, **%rsi** har værdien **0x3**, og at program tælleren PC har værdien **0x77774402**. Alle værdier er underforstået HEX, udlad derfor 0x prefix.

Stage	pushq %rsi
Fetch	$icode:ifun \leftarrow M_1[\dots] = \dots : \dots$ $rA:rB \leftarrow M_1[\dots] = \dots : \dots$ $valP \leftarrow \dots = \dots$
Decode	$valA \leftarrow R[\dots] = \dots$ $valB \leftarrow R[\%rsp] = \dots$
Execute	$valE \leftarrow valB + (-8) = \dots$
Memory	$M_8[\dots] \leftarrow \dots$
Write back	$R[\%rsp] \leftarrow \dots$
PC update	$PC \leftarrow \dots$

Eksekvering (udlad 0x prefix)

Trin	push %rsi
Fetch	$icode:ifun \leftarrow M1[\text{77774402}] = \text{a} : \text{0}$ $rA:rB \leftarrow M1[\text{77774403}] = \text{6} : \text{f}$ $valP \leftarrow \text{77774404}$
Decode	$valA \text{ 3 } \leftarrow R[\text{6}]$ $valB \text{ 348 } \leftarrow R[\text{4}]$
Execute	$valE \text{ 340 } \leftarrow valB + (-8)$

Execute	valE ← valB + (-8)	✓
---------	--------------------	---

Trin	push %rsi
Memory	$M_8[340] \leftarrow 3$ ✓
Writeback	$R[4] \leftarrow 340$ ✓
PC update	$PC \leftarrow 77774404$ ✓

Læs fig 4.20 og Aside p 428 i bogen, og tilhørende tekst. Bemærk den anvendte notation $R[]$, $M[]$.

Stage	pushq rA
Fetch	$icode:ifun \leftarrow M_1[PC]$ $rA:rB \leftarrow M_1[PC + 1]$ $valP \leftarrow PC + 2$
Decode	$valA \leftarrow R[rA]$ $valB \leftarrow R[\%rsp]$
Execute	$valE \leftarrow valB + (-8)$
Memory	$M_8[valE] \leftarrow valA$
Write back	$R[\%rsp] \leftarrow valE$
PC update	$PC \leftarrow valP$

Figure 4.20 Computations in sequential pushq and popq. These instructions push a

Eksekvering (udlad 0x prefix)

Trin	push %rsi
Fetch	icode:ifun <- M1[77774402] = a : 0 rA:rB <- M1[77774403] = 6 : f valP <- 77774404
Decode	valA 3 <- R[6] valB 348 <- R[4]
Execute	valE 340 <- valB + (-8)
Memory	M8[340] <- 3
Writeback	R[4] <- 340
PC update	PC <- 77774404

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 3:55 PM	Completed Saturday, 7 June 2025, 3:55 PM
Duration 44 secs	Marks 20.00/20.00	Grade 10.00 out of 10.00 (100%)

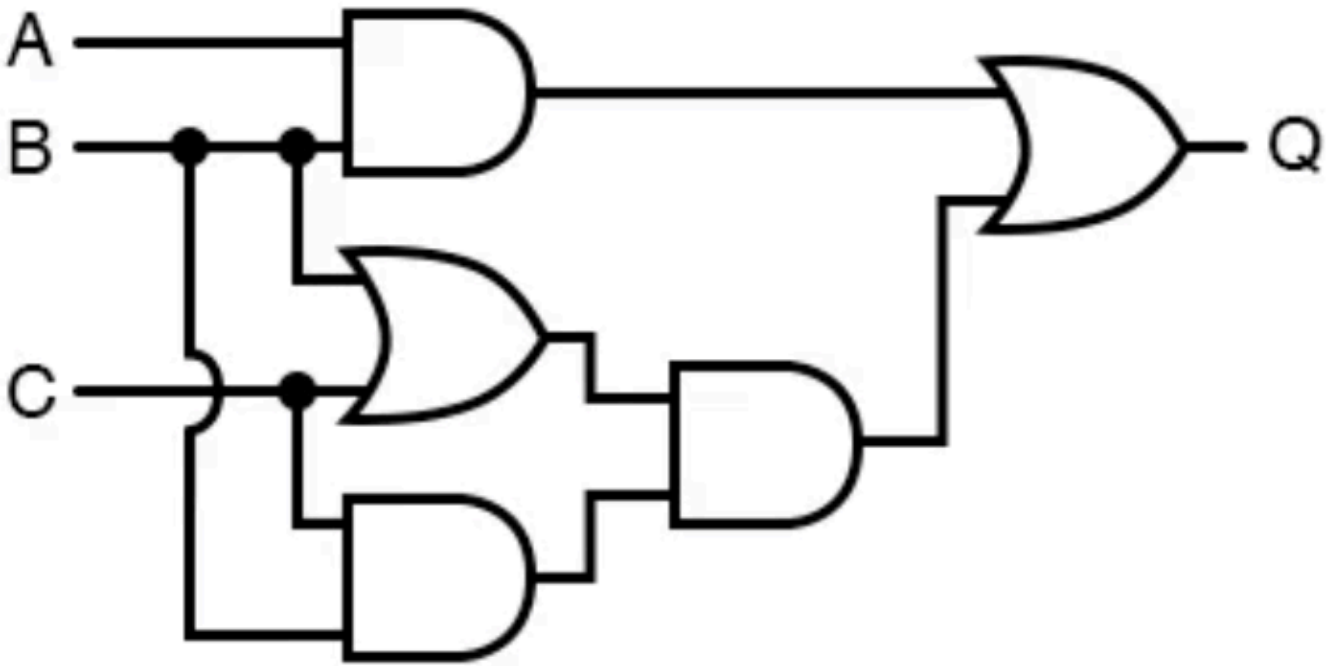
Question 1

Flag question

Correct

Mark 10.00 out of 10.00

Spørgsmålet omhandler kombinatoriske kredsløb og boolsk algebra. Betragt nedenstående kombinatoriske kredsløb:



Hvilke af nedenstående boolske udtryk er **ækvivalente** (giver samme resultat/output) med kredsløbets output Q?

(her anvendes notationen for boolske operatorer som vist på side 87 i lærebogen: & | ~ ^)

Select one or more:

- ☐ a. $A \& B \& C$
- ☒ b. $A \& B \mid B \& C \& B \& C$ ✓
- ☒ c. $A \& B \mid B \& C \& (B \mid C)$ ✓
- ☒ d. $B \& (A \mid C)$ ✓

Your answer is correct.

$A \& B \mid B \& C \& B \& C$ $A \& B \mid B \& C \& B \& C$ $B \& (A \mid C)$

The correct answers are: $A \oplus B \oplus C$

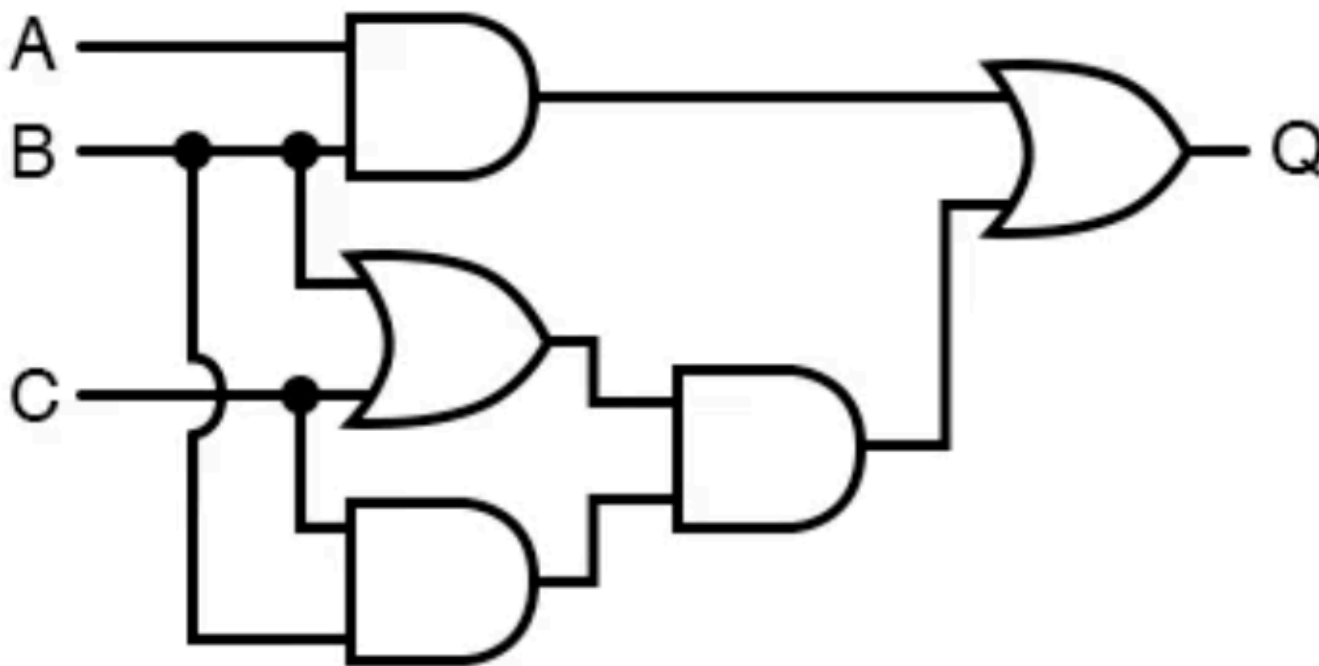
Question 2

Flag question

Correct

Mark 10.00 out of 10.00

Betrakt nedenstående kombinatoriske kredsløb:



Udfyld sandhedstabellen for kredsløbet.

Sandhedstabel

A	B	C	Q	
0	0	0	0	✓
0	0	1	0	✓
0	1	0	0	✓
0	1	1	1	✓
1	0	0	0	✓
1	0	1	0	✓
1	1	0	1	✓
1	1	1	1	✓



Status Finished	Started Saturday, 7 June 2025, 4:13 PM	Completed Saturday, 7 June 2025, 4:15 PM
Duration 1 min 30 secs	Marks 4.00/4.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Antag at en processor har en `add` instruktion med en latency på 2, og en Cycles Per Instruction (CPI) på 1.

Hvor mange clock cycles vil dette program tage at eksekvere?

```
addl %eax, %ebx  
addl %ecx, %edx  
addl %eax, %ebx
```

Select one:

- ☐ a. 2
- ☐ b. 3
- ☒ c. 4 ✓
- ☐ d. 5
- ☐ e. 6

Your answer is correct.

The correct answer is: 4

Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvad sker der ved en branch mis-prediction?

Select one:

- ☒ a. Programmet vil køre langsommere ✓
- ☐ b. Der vil ske en fejl
- ☐ c. Processoren regner forkert

Your answer is correct.

The correct answer is: Programmet vil køre langsommere

Question 3

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvad betyder disse begreber?

Branch prediction	Forudsigelse af om et jump bliver taget	✓
Pipelined	Instruktioners eksekvering overlapper	✓
Latency	Antal clock cycles det tager at færdiggøre en instruktion	✓
Out-of-order execution	Instruktioner færdiggøres ikke i program-rækkefølge	✓
Super-scalar	Mere end en instruktion af samme type kan færdiggøres på en clock-cycle	✓
Cycles Per Instruction (CPI)	Antal clock cykler det (i gennemsnit) tager at behandle en instruktion	✓

Your answer is correct.

The correct answer is: Branch prediction → Forudsigelse af om et jump bliver taget, Pipelined → Instruktioners eksekvering overlapper, Latency → Antal clock cycles det tager at færdiggøre en instruktion, Out-of-order execution → Instruktioner færdiggøres ikke i program-rækkefølge, Super-scalar → Mere end en instruktion af samme type kan færdiggøres på en clock-cycle, Cycles Per Instruction (CPI) → Antal clock cykler det (i gennemsnit) tager at behandle en instruktion

Question 4

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Hvilken af disse to maskiner er gennemsnitligt hurtigst?

1. Maskine: Clock frekvens på 4 GHz, CPI (Cycles Per Instruction) på 2, latency 2.
2. Maskine: Clock frekvens på 3 GHz, CPI (Cycles Per Instruktion) på 1, latency 3.

Select one:

- ☐ a. Maskine 1
- ☒ b. Maskine 2 

Your answer is correct.

Gennemsnitligt færddiggør maskinerne

1 / CPI

instruktioner per clock cycle.

Derfor færddiggør maskinerne ca.:

1. Maskine: udfører 4G cycles per sekund, og bruger 2 cycles i snit per instruktion: $1/2 * 4 \text{ GHz} = 2 \text{ G}$ instruktioner per sekund
2. Maskine: udfører 3G cycles per sekund og bruger 1 cycles i snit per instruktion; $1/1 * 3 \text{ GHz} = 3 \text{ G}$ instruktioner per sekund

Latency er kun en konstant forsinkelse på afslutningen af en instruktion, som kun har betydning for opstart af en ny instruktion, som er afhængig af resultatet fra den foregående; dette er normalt inkluderet i den benchmark der har givet CPI tallet.

The correct answer is: Maskine 2

[Finish review](#)



Status

Finished

Started

Saturday, 7 June 2025, 4:24 PM

Completed

Saturday, 7 June 2025, 4:47 PM

Duration

22 mins 55 secs

Grade

10.00 out of 10.00 (100%)

Question 1

Flag question

Correct

Mark 1.00 out of 1.00

Det oplyses at en processor har to floating point multiplikations enheder, der er fuldt pipelinede (1 cycle/issue) med en latenstid på 5 cykler.

Hvor mange floating point multiplikationer, kan processoren afslutte på 10 clock cykler, hvis der ikke er yderligere hardware begrænsninger (registre, hukommelsesindlæsning)?

Hint: Det kan være en hjælp med det stykke (digitalt) ternet papir og tegne et tidslinie diagram over hvordan instruktionerne kan afvikles (en afviklingsplan).

Answer:

12



Cyklus	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Mult unit 1	R0 = a*b													
		R2 = e*f												
			R4 = i*j											
				R6 = m*n										
					R8 = q*r									
						R10 = u*v								
Mult unit 2	R1 = c*d													
		R3 = g*h												
			R5 = k*l											
				R7 = o*p										
					R9 = s*t									
						R11 = x*y								

The correct answer is: 12

Question 2

Flag question

Correct

Mark 1.00 out of 1.00

Det oplyses at en processor har to floating point multiplikations enheder, der er fuldt pipelinede (1 cycle/issue) med en latenstid på 5 cykler.

Hvor mange floating point multiplikationer, kan processoren afslutte på 15 clock cykler, hvis der ikke er yderligere hardware begrænsninger (registre, hukommelsesindlæsning)?

Answer:

22



Cyklus	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
Mult unit 1	R0 = a*b																	
		R2 = e*f																
			R4 = i*j															
				R6 = m*n														
					R8 = q*r													
						R10 = u*v												
							R12											
								R14										
									R16									
										R18								
Mult unit 2																		

The correct answer is: 22

Question 3

Flag question

Correct Mark 2.00 out of 2.00

Det oplyses at en processor har to floating point multiplikations enheder, der er fuldt pipelinede (1 cycle/issue) med en latenstid på 5 cykler.

Processoren skal beregne en serie af floating point multiplikationer:

```
double a[N];
double r=((a[0]*a[1])*a[2])*a[3];
```

Hvor mange clock cykler vil det mindst tage at beregne udtrykket (re-associering ikke tilladt), hvis der ikke er yderligere hardware begrænsninger (registre, hukommelsesindlæsning)?

15

✓

Hvor mange clock cykler per element bruger den (CPE) ?

3.75

✓

$r = (((a[0] * a[1]) * a[2]) * a[3])$

15 (=5 cykler per element)

Cyklus	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Mult unit 1	r0 = a[0]*a[1]																
						r1 = r0*a[2]											
											r = r1*a[3]						

Question 4

Flag question

Correct

Mark 2.00 out of 2.00

Det oplyses at en processor har to floating point multiplikations enheder, der er fuldt pipelinede (1 cycle/issue) med en latenstid på 5 cykler.

Processoren skal beregne en serie af floating point multiplikationer:

```
double a[N];
double r=a[0]*a[1]*a[2]*a[3];
```

Det oplyses at datasættet kan beregnes korrekt, uafhængigt af operationernes rækkefølge (re-associering tilladt).

Hvor mange clock cykler vil det mindst tage at beregne udtrykket, hvis der ikke er yderligere hardware begrænsninger (registre, hukommelsesindlæsning)?

10



Hvor mange clock cykler per element bruger den (CPE) ?

2.5



$r=a[0]*a[1]*a[2]*a[3]$

10 (10/4 cykler per element=2,5

Cyklus	0	1	2	3	4	5	6	7	8	9	10
Mult unit 1	R0 = a[0]*a[1]										
Mult unit 2	R1 = a[2]*a[3]										

Question 5

[Flag question](#)

Correct

Mark 2.00 out of 2.00

Det oplyses at en processor har to floating point multiplikations enheder, der er fuldt pipelinede (1 cycle/issue) med en latenstid på 5 cykler.

Processoren skal beregne en serie af floating point multiplikationer:

```
double a[N];  
double r=a[0]*a[1]*a[2]*a[3]*a[4]*a[5]*a[6]*a[7];
```

Det oplyses at datasættet kan beregnes korrekt, uafhængigt af operationernes rækkefølge (re-associering tilladt).

Hvor mange clock cykler vil det mindst tage at beregne udtrykket, hvis der ikke er yderligere hardware begrænsninger (registre, hukommelsesindlæsning)?



Hvor mange clock cykler per element bruger den (CPE) ?



Hvor mange clock cykler vil det mindst tage at beregne udtrykket (re-associering tilladt):

$$r = a[0] * a[1] * a[2] * a[3] * a[4] * a[5] * a[6] * a[7]$$

$$r = (a[0] * a[1]) * (a[2] * a[3]) * (a[4] * a[5]) * (a[6] * a[7])$$

$$r = (R0 * R1) * (R2 * R3)$$

$$r = R4 * R5$$

16 = (16/8=2 cykler per element)

Cyklus	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Mult unit 1	R0 = a[0]*a[1]																				
		R2 = a[4]*a[5]																			
Mult unit 2																					

Question 6

[Flag question](#)

Correct

Mark 2.00 out of 2.00

Det oplyses at en processor har to floating point multiplikations enheder, der er fuldt pipelinede (1 cycle/issue) med en latenstid på 5 cykler.

Processoren skal beregne en serie af floating point multiplikationer:

```
double a[N];  
double r=a[0]*a[1]*a[2]*a[3]*a[4]*a[5]*a[6]*a[7]*a[8]*a[9]*a[10]*a[11]*a[12]*a[13]*a[14]*a[15];
```

Det oplyses at datasættet kan beregnes korrekt, uafhængigt af operationernes rækkefølge (re-associering tilladt).

Hvor mange clock cykler vil det mindst tage at beregne udtrykket, hvis der ikke er yderligere hardware begrænsninger (registre, hukommelsesindlæsning)?



Hvor mange clock cykler per element bruger den (CPE) ?

Giv svar med to decimaler.



Hvor mange clock cykler vil det mindst tage at beregne udtrykket (re-associering tilladt):

```
r=a[0]*a[1]*a[2]*a[3]*a[4]*a[5]*a[6]*a[7]* a[8]*a[9]*a[10]*a[11] * a[12]*a[13]*a[14]*a[15]
```

```
r=(a[0]*a[1]) * (a[2]*a[3]) * (a[4]*a[5]) * (a[6]*a[7])* (a[8]*a[9]) * (a[10]*a[11]) * (a[12]*a[13])
```

$$r = (R0 * R1) * (R2 * R3) * (R4 * R5) * (R6$$

= (R8 *R9 *R10 *R11

$$=R_{12} * R_{13}$$

23 = (23/16 = 1,4375 cykler per element)

[illegible]

Finish review



Status Finished	Started Saturday, 7 June 2025, 4:53 PM	Completed Saturday, 7 June 2025, 4:53 PM
Duration 11 secs	Marks 2.00/2.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Betragt følgende C program, som beregner summen af elementerne i et 2-D array.

```
long arr_sum ( long a [M] [N] ) {  
    long sum , i , j ;  
    for ( i =0; i < M; i ++ ) {  
        for ( j =0; j < N; j ++ ) {  
            sum += a [ i ] [ j ] ;  
        }  
    }  
}
```

Hvilke af programmets variable bliver brugt med god *spatial lokalitet*?

- ☐ a. i
- ☐ b. j
- ☒ c. a 
- ☐ d. sum
- ☐ e. arr_sum

Your answer is correct.

"a" læses i row-major rækkefølge (rækkevist med skridtlængde 1 long)

The correct answer is: **a**

Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Betragt følgende C program, som beregner summen af elementerne i et 2-D array.

```
long arr_sum ( long a [M] [N] ) {  
    long sum , i , j ;  
    for ( i =0; i <M; i ++ ) {  
        for ( j =0; j <N; j ++ ) {  
            sum += a [ i ] [ j ] ;  
        }  
    }  
}
```

Hvilke af programmets variable bliver brugt med god **temporal lokalitet**?

- ☒ a. sum
- ☐ b. a
- ☐ c. arr_sum
- ☒ d. j
- ☒ e. i

Your answer is correct.

sum læses og skrives i den indre løkke: god temp lokalitet

i og j læses i den indre løkke, og j skrives (j++)

ingen elementer i a genbruges

The correct answers are: **sum,i,j**



Status Finished	Started Saturday, 7 June 2025, 5:16 PM	Completed Saturday, 7 June 2025, 5:19 PM
Duration 3 mins 30 secs	Marks 25.00/25.00	Grade 10.00 out of 10.00 (100%)

Question 1

Flag question

Correct

Mark 1.00 out of 1.00

Et lille hukommelsessystem har 8 bit adresser og en tilhørende 2-vejs associativ cache med 8 sets og en blok størrelse på 8 bytes.

- Udfyld nedenstående tabel med angivelse af hvilke adresse bits, der skal bruges til hhv.
 - block offset (angiv med et 'B')
 - set index (angiv med et 'I')
 - tag (angiv med et 'T')

7	6	5	4	3	2	1	0

Adresse bit 0	B	✓
Adresse bit 1	B	✓
Adresse bit 2	B	✓
Adresse bit 3	I	✓
Adresse bit 4	I	✓
Adresse bit 5	I	✓
Adresse bit 6	T	✓
Adresse bit 7	T	✓

Your answer is correct.

Block størrelsen er givet til 8 bytes: der skal bruges 3 bit til at skelne hvilket offset (adresserne 0–7) inden for blokken, der ønskes. Disse 3 er de 3 mindstbetydende bits af adressen: bits 0–2

Der er 8 sets: der skal bruges 3 bits til at skelne imellem hvilket set (set nummereret 0 ... 7; på binær 000 ... 111) der skal bruges. Disse er således i eksemplet bits 3–5.

Resterende (mest betydende bits) er cache tag. Disse bits er i eksempel adressen så bits 6 og 7.

The correct answer is: **Adresse bit 0 → B, Adresse bit 1 → B, Adresse bit 2 → B, Adresse bit 3 → I, Adresse bit 4 → I, Adresse bit 5 → I, Adresse bit 6 → T, Adresse bit 7 → T**

Question 2

Flag question

Correct

Mark 24.00 out of 24.00

Et lille hukommelsessystem har 8 bit adresser og en tilhørende 2-vejs associativ cache med 8 sets og en blok størrelse på 8 bytes.

I det følgende er vist en 2-vejs associative cache med 8 sets og en blokstørrelse på 8 bytes (byte med offset 0 er vist til venstre). Adresser er 8 bit. Antag i tilfælde af cache miss at den pågældende cache line fyldes med værdien 0xAABBCCDDAABBCCDD.

	Line 1									Line 2										
Set	V	Tag	Value (Hex)								V	Tag	Value (Hex)							
0	0	3	12	13	14	15	A0	A1	A2	A3	1	2	C1	C2	C3	C4	C5	C6	B6	C6
1	1	2	A0	B0	C0	D0	98	97	80	81	1	3	12	23	34	45	12	34	56	78
2	1	3	00	00	00	00	B9	B1	B2	B8	1	2	76	65	54	43	65	65	87	86
3	1	0	00	00	00	00	00	01	02	03	1	3	00	01	02	03	00	00	00	00
4	0	3	22	23	24	25	B0	B1	B2	B3	1	2	BF	BE	BD	BC	BB	BA	B9	B8
5	1	2	A1	B1	C1	D1	88	87	86	85	1	3	11	11	11	11	01	01	01	01
6	0	3	A2	B2	B7	B8	A3	B3	B7	C1	1	2	E6	E5	E4	E3	E5	E5	E7	E6
7	1	0	FA	FB	FC	FD	FE	F1	F2	F3	1	3	D8	D9	DA	DB	AC	DC	DE	DF

Udfyld nedenstående tabel når cache-systemet læser en byte for den angivne sekvens af referencer (adresser). Angiv alle værdier som hex-tal (uden 0x prefix).

Cache udlæsning

Reference Nr.	Adresse	Tag	Set Index	Offset	Udlæst værdi
1	8B	2 ✓	1 ✓	3 ✓	d0 ✓
2	B7	2 ✓	6 ✓	7 ✓	e6 ✓
3	E5	3 ✓	4 ✓	5 ✓	bb ✓
4	4E	1 ✓	1 ✓	6 ✓	cc ✓
5	E1	3 ✓	4 ✓	1 ✓	bb ✓
6	FF	3 ✓	7 ✓	7 ✓	df ✓

Adresse	Binary	Tag	Set	Offset	Værdi
0x8B	10.00 1.011	2	1	3	D0
0xB7	10.11 0.111	2	6	7	E6
0xE5	11.10 0.101	3	4	5	BB (miss invalid bit)
0x4E	01.00 1.110	1	1	6	CC (miss tag)
0xE1	11.10 0.001	3	4	1	BB (replenished)
0xFF	11.11 1.111	3	7	7	DF

Bruger besvarelsen forkert byteorder er resultatet: 98,E6,CC,BB,CC,D8

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 5:27 PM	Completed Saturday, 7 June 2025, 5:27 PM
Duration 12 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

Question 1

Flag question

Correct

Mark 1.00 out of 1.00

Betragt nedenstående unix program:

```
int main() {
    int child_status;
    printf("CAOS\n");
    if (fork() == 0) {
        printf("Hej\n");
    } else {
        printf("med\n");
        wait(&child_status);
        printf("dig!\n");
    }
    printf("Bye\n");
}
```

Hvilke afviklinger kan programmet generere? Antag i opgaven at printf udføres atomisk.

Afviklinger

A	B	C	D	E	F
CAOS	CAOS	CAOS	CAOS	CAOS	CAOS
Hej	Hej	Hej	Hej	med	med
med	Bye	med	med	Hej	Bye
dig!	med	Bye	dig!	Bye	Hej
Bye	dig!	dig!	Bye	dig!	dig!
	Bye	Bye	Bye	Bye	Bye

☐ a. A

☒ b. B ✓

☒ c. C ✓

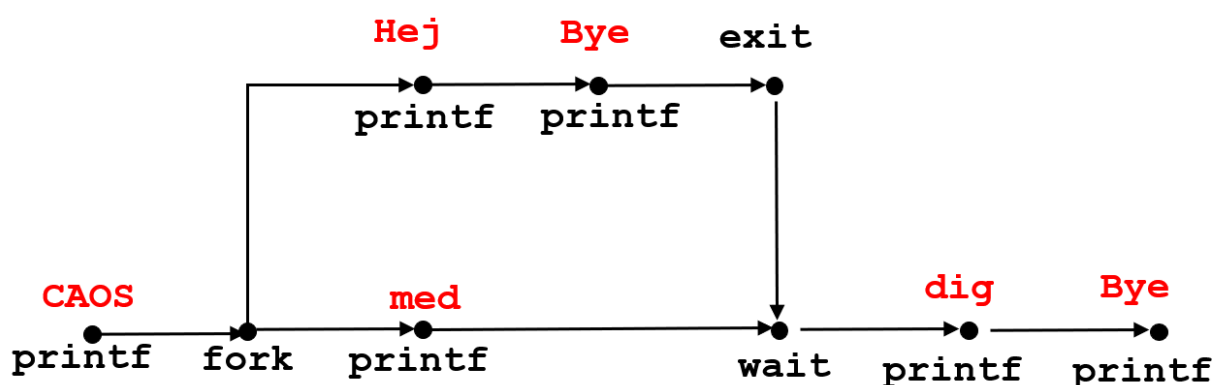
☐ d. D

✓ e. E ✓

■ f. F

Your answer is correct.

Lav som det første programmets process-graf model, som afspejler programmets sekventielle afhængigheder, og hvad der er sideordnet/concurrent.



Husk

at sideordnede (concurrent) hændelser i forskellige proceser kan "sammenflettes" på alle mulige måder (udfaldet afhænger af timingen i systemet og schedulerens valg, og kan dermed ikke forudsiges).

Wait blokerer indtil et af processens børn terminerer (hvis en ikke allerede er) og synkroniserer (ordner) dermed hændelserne i to processer.

at fork() skaber en kopi af processen (incl dens adresserum); dermed får både parent og child sin egen kopi af eventuelle variable. Forskel er returværdien fra fork()!

The correct answers are: **B,C,E**

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 5:28 PM	Completed Saturday, 7 June 2025, 5:31 PM
Duration 2 mins 53 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

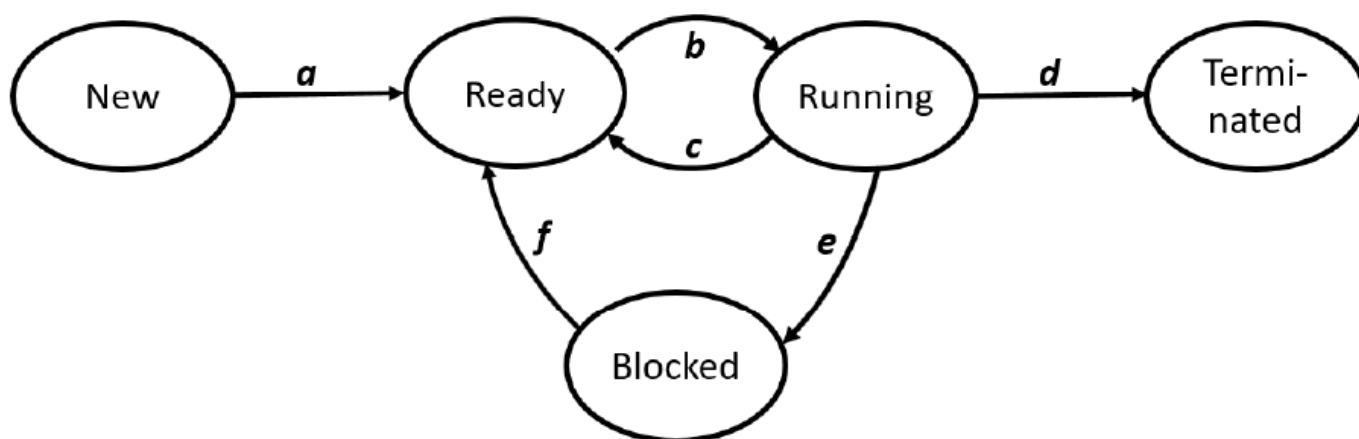
Question 1

Flag question

Correct

Mark 1.00 out of 1.00

Betrægt trådtilstandsmodellen i nedenstående figur.



Angiv med bogstaverne a – f den tilstandsovergang, der svarer til nedenstående situationer:

Processen kalder en længerevarende synkron I/O operation

e



Processen kalder wait, men ingen af dens childs er terminerede.

e



En kørende process afbrydes af et timer interrupt, som angiver, at dens timeslice er udløbet

c



Processen kalder exit

d



Pagefault indtræffer ved adgang til en allokeret side, men som ikke er resident i RAM.

e



Data fra længerevarende synkron I/O operation er nu klar til processen

f



Your answer is correct.

I **New** forberedes processen (fx alokering og initialisering af hukommelse og kernen's datastrukturer).

i **Ready** er processen klar til afvikling, når CPU'en bliver ledig, og scheduleren vælger den.

I **Running** afvikler processen aktivt på CPU'en

i **Blocked** kan processen ikke fortsætte, da den afventer en resource eller i/i svar.

I **Terminated** kan processen ikke længere afvikles (hukommelse frigives, evt. efterlades kun en status).

- Et kørende program (dernet i Running) hvis timeslice udløber via et timerinterrupt bliver sat i Ready (kan afvikles så snart CPU bliver ledig og dens tur)
- Wait kaldet (dermed i Running) blokkerer processen indtil et child er termineret.
- Ved længere varende i/o operationer (fx læsning fra en fil på disk) kan processen ikke fortsætte før data fra i/o enheden er klar til levering til CPU; dermed blokkerer OS processen. I den mellemliggende periode afvikles andre processer.
- Når data er klar signalerer i/o enheden dette til CPU via et interrupt, så den flytter den blokkerede process til Ready (hvorfra den kan afvikles når det bliver dens tur).
- Exit kaldes af processen (i Running) som terminerer processen (deallokerer resourcer).
- Når en pagefault (exception) indtræffer ved adgang til en allokeret side, men som ikke er resident i RAM, skal den manglende side indlæses fra disk, som er en længerevarende i/o operation; dermed blokeres processen, indtil læsningen er færdig (hvorefter den flyttes til Ready).

The correct answer is:

Processen kalder en længerevarende synkron I/O operation → e,

Processen kalder wait, men ingen af dens childs er terminerede. → e,

En kørende process afbrydes af et timer interrupt, som angiver, at dens timeslice er udløbet → c,

Processen kalder exit → d,

Pagefault indtræffer ved adgang til en allokeret side, men som ikke er resident i RAM. → e,

Data fra længerevarende synkron I/O operation er nu klar til processen → f

[Finish review](#)



Status

Finished

Started

Saturday, 7 June 2025, 5:39 PM

Completed

Saturday, 7 June 2025, 5:40 PM

Duration

1 min 18 secs

Marks

22.00/22.00

Grade

10.00 out of 10.00 (100%)

Question 1

Flag question

Correct Mark 22.00 out of 22.00

Et operativ system, som kører på en enkelt-kerne maskine, afvikler 2 processer med en Round Robin Scheduler med et time slice på 100 ms. To processer, P1 og P2, er blevet startet og er klar til afvikling. For at afslutte deres beregninger og terminere, kræver P1 CPU'en i 400 ms totalt, og P2 beregner i 200ms, hvorefter den laver i/o i 300ms, og afslutter med endnu en beregning på 100 ms. Givet at P1 starter i running-tilstanden, vis hvordan processerne afvikles og angiv for hver tidsperiode deres tilstand (iht. 5-tilstands-modellen):

Schedule og process tilstande

Tid (ms)	0	100	200	300	400	500	600	700	800	900
P1	Running ✓	Ready ✓	Running ✓	Ready ✓	Running ✓	Running ✓	Terminated ✓	Terminated ✓	Terminated ✓	Ter ✓
P2	Ready ✓	Running ✓	Ready ✓	Running ✓	Blocked ✓	Blocked ✓	Blocked ✓	Running ✓	Terminated ✓	Ter ✓

Et operativ system, som kører på en enkelt-kerne maskine, afvikler 2 processer med en Round Robin Scheduler med et time slice på 100 ms. To processer, P1 og P2, er blevet startet og er klar til afvikling. For at afslutte deres beregninger og terminere, kræver P1 CPU'en i 400 ms totalt, og P2 beregner i 200ms, hvorefter den laver i/o i 300ms, og afslutter med endnu en beregning på 100 ms. Givet at P1 starter i running-tilstanden, vis hvordan processerne afvikles og angiv for hver tidsperiode deres tilstand (iht. 5-tilstands-modellen):

Schedule og process tilstande

Tid (ms)	0	100	200	300	400	500	600	700	800	900	1000
P1	Running ↕	Ready ↕	Running ↕	Ready ↕	Running ↕	Running ↕	Terminated ↕	Terminated ↕	Terminated ↕	Terminated ↕	Terminated ↕
P2	Ready ↕	Running ↕	Ready ↕	Running ↕	Blocked ↕	Blocked ↕	Blocked ↕	Running ↕	Terminated ↕	Terminated ↕	Terminated ↕

En enkelt CPU/kerne maskine kan kun have én process i Running ad gangen!

0-99: P1 oplyses til at være i Running (dermed afvikle på CPU'en); P2 oplyses til at være Ready.

100-199: P1's time slice er netop udløbet, og OS vælger en anden Ready process ("Round Robin: efter tur princip"); dermed bliver P2 Running, og P1 Ready.

200-299 P2's timeslice udløber; P1 vælges til Running

300-399: Igen, modsat. P2 Running

400-499: P2 har nu beregnet samlet 200 ms, og har netop igangsætte en I/O operation. Dermed blokeres P2. Kun P1 er Ready, så denne aktiveres.

500-599 I/O operationen fortsætter. Med kun disse 2 processer er det kun P1, der kan afvikles, som dermed fortsat får CPU'en.

600-699: P1 har beregnet til ende (ialt 400 ms) og terminerer dermed (og forbliver termineret!). P2 er stadig blokeret. Bemærk her, at CPU'en nu er IDLE (ingen Ready (køreklare) processer).

700-799: Interrupt signalerer, at P2's I/O operation er færdig. OS flytter P2 til 'Ready', men da dette nu er den eneste Ready process sættes den umiddelbart til "Running".

800- 800: P2 har nu også beregnet i den krævede tid og terminerer.

900- Begge er terminerede (og forbliver terminerede)!

Finish review



Status	Started	Completed	Duration	Marks
Finished	Saturday, 7 June 2025, 5:44 PM	Saturday, 7 June 2025, 5:56 PM	12 mins 46 secs	261.00/261.00
Grade	10.00 out of 10.00 (100%)			

Question 1

Flag question

Correct

Mark 87.00 out of 87.00

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Vis hvordan den virtuelle adresse **0x150A** oversættes til fysisk adresse. Brug minus symbolet "-" til at angive at en værdi ikke kan genereres.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	0	0	0	1	0	1	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Adresse oversættelse

Parameter	Værdi
VPN (0x...):	A8 ✓
TLB Indeks (0x...):	0 ✓
TLB-Tag (0x...):	2A ✓
TLB-Hit (j/n):	n ✓
Page fault (j/n):	n ✓
PPN (0x...):	06 ✓

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	0	0	1	0	1	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Sidestørrelse på 32 bytes betyder at VPO er 5 bits. Så bits 0-4 bruges til VPO, bits 5-13 VPN.

TLB-I kræver 2 bits da TLB cache har 4 sets. TLB-I findes i bits 5+6. TLB-T findes i bits 7-13.

Bits i virtuel adresse

VPN							VPO						
13	12	11	10	9	8	7	6	5	4	3	2	1	0
TLB-Tag							TLB-Index						
0	1	0	1	0	1	0	0	0	0	1	0	1	0

Bits i fysisk adresse

PPN							VPO				
11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	0	0	1	0	1	0

Question 2

Flag question

Correct

Mark 87.00 out of 87.00

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Vis hvordan den virtuelle adresse **0x1590** oversættes til fysisk adresse. Brug minus symbolet "-" til at angive at en værdi ikke kan genereres.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	0	0	1	0	0	0	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Adresse oversættelse

Parameter	Værdi
VPN (0x...):	AC ✓
TLB Indeks (0x...):	0 ✓
TLB-Tag (0x...):	2B ✓
TLB-Hit (j/n):	j ✓
Page fault (j/n):	n ✓
PPN (0x...):	4A ✓

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	1	0	1	0	0	0	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Sidestørrelse på 32 bytes betyder at VPO er 5 bits. Så bits 0-4 bruges til VPO, bits 5-13 VPN.

TLB-I kræver 2 bits da TLB cache har 4 sets. TLB-I findes i bits 5+6. TLB-T findes i bits 7-13.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	0	0	1	0	0	0	0

Bits i fysisk adresse PPN

11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	1	0	1	0	0	0	0

Question 3

Flag question

Correct

Mark 87.00 out of 87.00

I denne opgave skal du oversætte virtuelle til fysiske adresser i nedenstående mini hukommelsesystem givet ved:

- Sidestørrelsen er 32 bytes
- Virtuelle adresser er 14 bits lange
- Fysiske adresser er 12 bits lange
- Der er tilknyttet en 3-vejs TLB (translation lookaside buffer) med 4 sets. Hukommelsen kan udlæses byte-vist.
- Sidetabel og TLB har følgende indhold:

VPN	PPN	Valid	VPN	PPN	Valid
00	7F	1	A8	06	1
01	6E	1	A9	17	1
02	5D	1	AA	28	0
03	4C	1	AB	39	1
04	3B	1	AC	4A	1
05	2A	1	AD	5B	1
06	19	1	AE	01	1
07	08	1	AF	00	0

Set	Tag	PPN	Valid	Tag	PPN	Valid	Tag	PPN	Valid
0	01	3B	0	2B	4A	1	2A	06	0
1	2A	17	1	01	2A	0	00	6E	1
2	00	5D	0	01	19	1	2B	01	1
3	2A	39	1	2E	BC	0	2B	00	0

Vis hvordan den virtuelle adresse **0x15EA** oversættes til fysisk adresse. Brug minus symbolet "-" til at angive at en værdi ikke kan genereres.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	1	1	0	1	0	1	0
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Adresse oversættelse

Parameter	Værdi
VPN (0x...):	AF ✓
TLB Indeks (0x...):	3 ✓
TLB-Tag (0x...):	2B ✓
TLB-Hit (j/n):	n ✓
Page fault (j/n):	j ✓
PPN (0x...):	- ✓

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	-	-	-	-	-	-	-	-
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Sidestørrelse på 32 bytes betyder at VPO er 5 bits. Så bits 0-4 bruges til VPO, bits 5-13 VPN.

TLB-I kræver 2 bits da TLB cache har 4 sets. TLB-I findes i bits 5+6. TLB-T findes i bits 7-13.

Når der er sidefejl, kan der ikke generes nogen fysisk adresse (og dermed heller intet PPN). Dert er dermed en fejl at forsøge at generere fysisk adresse.

Bits i virtuel adresse

13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	1	1	1	0	1	0	1	0

Bits i fysisk adresse

11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	-	-	-	-	-	-	-	-

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 5:58 PM	Completed Saturday, 7 June 2025, 6:03 PM
Duration 4 mins 12 secs	Marks 4.00/4.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

The following program uses 3 threads to find the maximum element of an array by dividing it into 3 chunks processed by each thread.

Is the program correct?

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<pthread.h>
4  #include"common_threads.h"
5
6  //Number of worker threads
7  #define W 3
8  //Number of elements to search (a multiple of W for simplicity)
9  #define N (W*3)
10 //each worker thread each checks a chunk of elements
11 #define chunkSZ (N/W)
12
13 long arr[N]={1,8,2, 7,3,4, 5,6,9};
14 long max=0;
15
16 void* max_worker(void* arg){
17     long id=(long)arg;
18     for(int i=0;i<chunkSZ;i++) { //each worker process chunkSZ elements
19         long elem=arr[id*chunkSZ+i];
20         if(elem>max) max=elem;
21     }
22     return NULL;
23 }
24
25 int main(int argc,char argv[]){
26
27     pthread_t t1, t2,t3;
28     Pthread_create(&t1,NULL,max_worker,(void*)0);
29     Pthread_create(&t2,NULL,max_worker,(void*)1);
30     Pthread_create(&t3,NULL,max_worker,(void*)2);
31     Pthread_join(t1,NULL);
32     Pthread_join(t2,NULL);
33     Pthread_join(t3,NULL);
34
35     printf("Max %ld\n", max);
```

```
36   return 0;  
37 }
```

- ☐ a. Yes, as the threads checks separate portions/chunks of the array
- ☒ b. No, It contains a race condition. ✓
- ☐ c. No, It contains a deadlock
- ☐ d. Yes, as the threads does not run concurrently

Your answer is correct.

Analyse the program to determine which resources (variables and memory locations in this case) are shared (read or written by more than one thread) between the threads.

May these resources be accessed read/written concurrently? If so, the program may have a potential race-condition (the result depends on the relative speed in which the threads happens to be executed).

The correct answer is: **No, It contains a race condition.**

Question 2

[Flag question](#)

Correct

Mark 2.00 out of 2.00

The following program uses 3 threads to find the maximum element of an array.

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<pthread.h>
4  #include"common_threads.h"
5
6  //Number of worker threads
7  #define W 3
8  //Number of elements to search (a multiple of W for simplicity)
9  #define N (W*3)
10 //each worker thread each checks a chunk of elements
11 #define chunkSZ (N/W)
12
13 long arr[N]={1,8,2, 7,3,4, 5,6,9};
14 long max=0;
15
16 void* max_worker(void* arg){
17     long id=(long)arg;
18     for(int i=0;i<chunkSZ;i++) { //each worker process chunkSZ elements
19         long elem=arr[id*chunkSZ+i];
20         if(elem>max) max=elem;
21     }
22     return NULL;
23 }
24
25 int main(int argc,char argv[]){
26
27     pthread_t t1, t2,t3;
28     Pthread_create(&t1,NULL,max_worker,(void*)0);
29     Pthread_create(&t2,NULL,max_worker,(void*)1);
30     Pthread_create(&t3,NULL,max_worker,(void*)2);
31     Pthread_join(t1,NULL);
32     Pthread_join(t2,NULL);
33     Pthread_join(t3,NULL);
34
35     printf("Max %ld\n", max);
36     return 0;
```


37

}

Which line number is the first in its critical section ?



Which line number is the last in critical section ?



The critical resource is the "max" variable. The critical section is the if-then statement in line 20. If the thread is preempted after comparing, but before writing, another thread may have found another (better) maximum value, that becomes overwritten:

EX max=5;

e1=7, e2=9

thread1	thread2
---------	---------

<pre>if(e1>5) if(e2>5) max=9 max=5</pre>	
--	--

Question 3

Flag question

Correct

Mark 1.00 out of 1.00

Complete the following program with proper synchronization such that it correctly prints the sum of the lengths of the strings.

```
#include<stdio.h>
#include<string.h>
#include <pthread.h>
#include "include/common_threads.h"
int totalLen=0;

pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;

void* mythread(void*arg){
    int len=strlen((char*)arg);
    Pthread_mutex_lock(&lock);
    totalLen+=len;
    Pthread_mutex_unlock(&lock);
    return NULL;
}

int main (int argc, char *argv[]) {
    pthread_t t1, t2;
    printf("main: begin\n");
    Pthread_create(&t1, NULL, mythread, "Hi, CAOS - Concurrency is fun");
    Pthread_create(&t2, NULL, mythread, "but also has numerous pitfalls!");
    /* empty */
    Pthread_join(t1, NULL);
    Pthread_join(t2, NULL);
    printf("main: Total Length=%d\n",totalLen);
    return 0;
}
```

pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;

Pthread_mutex_lock(&lock);

Pthread_mutex_unlock(&lock);

Pthread_mutex(&unlock);

printf("main: Total Length=%d\n",totalLen);

Pthread_cond_wait(&empty, &lock);

```
/* empty */
```

```
Pthread_mutex(&lock);
```

Your answer is correct.

The correct answer is: Complete the following program with proper synchronization such that it correctly prints the sum of the lengths of the strings.

```
#include<stdio.h>
#include<string.h>
#include <pthread.h>
#include "include/common_threads.h"
int totalLen=0;
[pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;]
void* mythread(void*arg){
    int len=strlen((char*)arg);
    [Pthread_mutex_lock(&lock);]
    totalLen+=len;
    [Pthread_mutex_unlock(&lock);]
    return NULL;
}
int main (int argc, char *argv[]) {
    pthread_t t1, t2;
    printf("main: begin\n");
    Pthread_create(&t1, NULL, mythread, "Hi, CAOS - Concurrency is fun");
    Pthread_create(&t2, NULL, mythread, "but also has numerous pitfalls!");
    [/* empty */]
    Pthread_join(t1, NULL);
    Pthread_join(t2, NULL);
    [printf("main: Total Length=%d\n",totalLen);]
    return 0;
}
```

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 6:04 PM	Completed Saturday, 7 June 2025, 6:05 PM
Duration 1 min 13 secs	Marks 1.00/1.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Betragt producer-consumer-problemet. Hvilke af nedenstående version er en korrekt implementering af produceren ved brug af semaforer?

```
#include "common_threads.h"

sem_t tom, fuld, mutex;
pthread_mutex_t mutex_lock;
pthread_cond_t fill, empty;
int loops, count, MAX;
init(){//...
}

void *producer_v1(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&tom);
        sem_wait(&mutex);
        put(i);
        sem_post(&mutex);
        sem_post(&fuld);
    }
}

void *producer_v2(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex_lock);
        while (count == MAX)
            Pthread_cond_wait(&fill, &mutex);
        put(i);
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex_lock);
    }
}

void *producer_v3(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex_lock);
        while (count == MAX)
            Pthread_cond_wait(&fill, &mutex_lock);
        put(i);
        sem_post(&empty);
        sem_post(&mutex_lock);
    }
}

void *produce_v4(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_cond_wait(&tom);
        Pthread_cond_wait(&mutex);
        put(i);
        Pthread_cond_signal(&mutex);
        Pthread_cond_signal(&fuld);
    }
}
```

Select one or more:

- ☒ 1. producer_v1 ✓
- ☐ 2. producer_v2
- ☐ 3. producer_v3
- ☐ 4. producer_v4

Your answer is correct.

Producer 1 is correct;

Producer 2 correct, but uses locks+conds, not not the requested semaphores.

Producer 3+4 uses wrong operations on the types.

The correct answer is: **producer_v1**

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 6:07 PM	Completed Saturday, 7 June 2025, 6:09 PM
Duration 1 min 25 secs	Marks 2.00/2.00	Grade 10.00 out of 10.00 (100%)

Question 1

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Nedenstående program simulerer et system bestående af 6 tråde: 3 pushere og 3 rygere. For at ryge en cigaret kræver det 3 ressourcer: en portion tobak, et stykke cigaret papir, og en (engangs-)lighter. Hver ryger har tilstrækkeligt af én af ressourcerne, men skal købe de to andre hos pusherne. Ressourcerne er modelleret som semaforer.

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include"common_threads.h"

sem_t tobak;
sem_t papir;
sem_t lighter;
sem_t tobak_penge;
sem_t papir_penge;
sem_t lighter_penge;

void * tobak_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&tobak);
        sem_wait(&tobak_penge);
    }
}
void * papir_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&papir);
        sem_wait(&papir_penge);
    }
}
void * lighter_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&lighter);
        sem_wait(&lighter_penge);
    }
}

void * ryger_med_tobak(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&papir);
        sem_wait(&lighter);
        printf("%ld ryger cigaret %d \n", (long)arg, i);
        sem_post(&papir_penge);
        sem_post(&lighter_penge);
    }
    printf("%ld ryger død\n", (long)arg);
}
void * ryger_med_papir(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&lighter);
        sem_wait(&tobak);
        printf("%ld ryger cigaret %d\n", (long)arg,i);
        sem_post(&tobak_penge);
        sem_post(&lighter_penge);
    }
    printf("%ld ryger død\n", (long)arg);
}

void * ryger_med_lighter(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&tobak);
```

```
sem_wait(&papir);
printf("%ld ryger cigaret %d \n", (long)arg,i);
sem_post(&papir_penge);
sem_post(&tobak_penge);
}
printf("%ld ryger død\n", (long)arg);
}

int main(int argc, char argv[]){
sem_init(&tobak, 0, 0);
sem_init(&papir, 0, 0);
sem_init(&lighter, 0, 0);
sem_init(&tobak_penge, 0, 0);
sem_init(&papir_penge, 0, 0);
sem_init(&lighter_penge, 0, 0);

pthread_t tp, lp, pp, rl, rt, rp;
Pthread_create(&tp, NULL, tobak_pusher, (void*)0);
Pthread_create(&lp, NULL, lighter_pusher, (void*)1);
Pthread_create(&pp, NULL, papir_pusher, (void*)2);
Pthread_create(&rl, NULL, ryger_med_lighter, (void*)3);
Pthread_create(&rt, NULL, ryger_med_tobak, (void*)4);
Pthread_create(&rp, NULL, ryger_med_papir, (void*)5);

Pthread_join(tp, NULL); Pthread_join(lp, NULL);
Pthread_join(pp, NULL); Pthread_join(rl, NULL);
Pthread_join(rt, NULL); Pthread_join(rp, NULL);
printf("All Dead\n");
return 0;
}
```

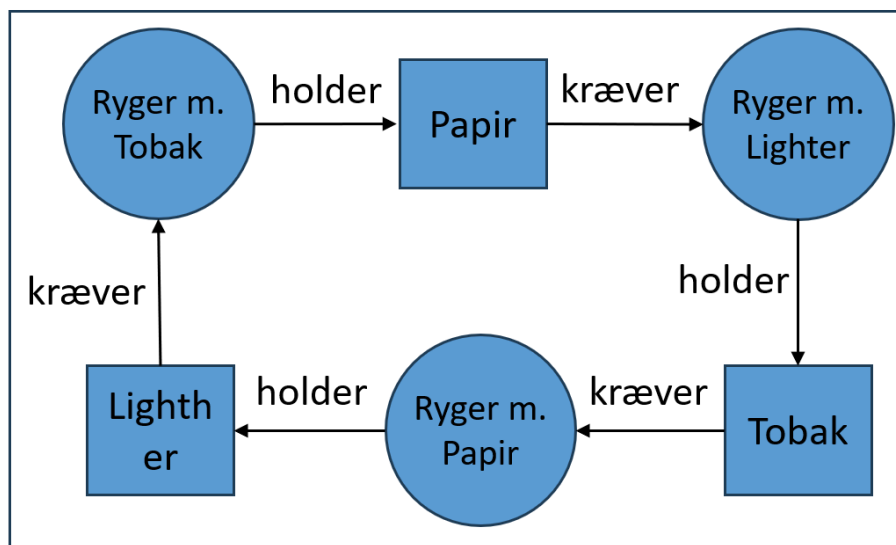
- ☐ a. Programmet er korrekt, da pusherne laver tilstrækkeligt mange resourcer
- ☒ b. Programmet er forkert, da det indeholder en deadlock 
- ☐ c. Programmet er forkert, da det indeholder en livelock
- ☐ d. Programmet er forkert, da det indeholder en race-condition
- ☐ e. Programmet er korrekt, da det bruger semaforer til at synkronisere trådene

Your answer is correct.

Deadlock scenarie:

Tid	Ryger med tobak	Ryger m papir	Ryger med lighter:
1	Wait(papir)		
2		wait(lighter)	
3			Wait(tobak)

4	Wait(lighter)		
5		wait(tobak)	
6			Wait(papir)



The correct answer is: **Programmet er forkert, da det indeholder en deadlock**

Question 2

[Flag question](#)

Correct

Mark 1.00 out of 1.00

Nedenstående program simulerer et system bestående af 6 tråde: 3 pushere og 3 rygere. For at ryge en cigaret kræver det 3 ressourcer: en portion tobak, et stykke cigaret papir, og en (engangs-)lighter. Hver ryger har tilstrækkeligt af én af ressourcerne, men skal købe de to andre hos pusherne. Ressourcerne er modelleret som semaforer.

Hjælp rygerne ved at gøre programmet færdigt, så alle rygerne "dør".

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include"common_threads.h"

sem_t tobak;
sem_t papir;
sem_t lighter;
sem_t tobak_penge;
sem_t papir_penge;
sem_t lighter_penge;

void * tobak_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&tobak);
        sem_wait(&tobak_penge);
    }
}
void * papir_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&papir);
        sem_wait(&papir_penge);
    }
}
void * lighter_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&lighter);
        sem_wait(&lighter_penge);
    }
}

void * ryger_med_tobak(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&papir);
        sem_wait(&lighter);
        printf("%ld ryger cigaret %d \n", (long)arg, i);
        sem_post(&papir_penge);
        sem_post(&lighter_penge);
    }
    printf("%ld ryger død\n", (long)arg);
}
void * ryger_med_papir(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&lighter);
        sem_wait(&tobak);
        printf("%ld ryger cigaret %d\n", (long)arg,i);
        sem_post(&tobak_penge);
        sem_post(&lighter_penge);
    }
    printf("%ld ryger død\n", (long)arg);
}

void * ryger_med_lighter(void *arg){
    for(int i=0;i<1000;i++){
```

```

sem_wait(&papir);
sem_wait(&tobak);
printf("%ld ryger cigaret %d \n", (long)arg,i);
sem_post(&papir_penge);
sem_post(&tobak_penge);
}
printf("%ld ryger død\n", (long)arg);
}

int main(int argc,char argv[]){
    sem_init(&tobak, 0, 0);
    sem_init(&papir, 0, 0);
    sem_init(&lighter, 0, 0);
    sem_init(&tobak_penge, 0, 0);
    sem_init(&papir_penge, 0, 0);
    sem_init(&lighter_penge, 0, 0);

    pthread_t tp, lp,pp, rl,rt,rp;
    Pthread_create(&tp,NULL,tobak_pusher,(void*)0);
    Pthread_create(&lp,NULL,lighter_pusher,(void*)1);
    Pthread_create(&pp,NULL,papir_pusher,(void*)2);
    Pthread_create(&rl,NULL,ryger_med_lighter,(void*)3);
    Pthread_create(&rt,NULL,ryger_med_tobak,(void*)4);
    Pthread_create(&rp,NULL,ryger_med_papir,(void*)5);

    Pthread_join(tp,NULL); Pthread_join(lp,NULL);
    Pthread_join(pp,NULL); Pthread_join(rl,NULL);
    Pthread_join(rt,NULL); Pthread_join(rp,NULL);
    printf("All Dead\n");
    return 0;
}

```

/* empty */

sem_wait(&tobak_penge);

sem_wait(&lighter_penge);

sem_wait(&lighter);

sem_wait(&papir_penge);

sem_post(&tobak_penge);

sem_wait(&tobak);

sem_wait(&papir);

sem_post(&papir_penge);

sem_post(&lighter_penge);

Your answer is correct.

Potentialet for deadlock kan forebygges ved at bryde muligheden for cirkulær venten ved at vedtage en total låse-orden, som alle tråde skal respektere; her fx: papir før lighter før tobak.

The correct answer is: Nedenstående program simulerer et system bestående af 6 tråde: 3 pushere og 3 rygere. For at ryge en cigaret kræver det 3 ressourcer: en portion tobak, et stykke cigaret papir, og en (engangs-)lighter. Hver ryger har tilstrækkeligt af éen af ressourcerne, men skal købe de to andre hos pusherne. Ressourcerne er modelleret som semaforer.

Hjælp rygerne ved at gøre programmet færdigt, så alle rygerne "dør".

```
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include"common_threads.h"

sem_t tobak;
sem_t papir;
sem_t lighter;
sem_t tobak_penge;
sem_t papir_penge;
sem_t lighter_penge;

void * tobak_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&tobak);
        sem_wait(&tobak_penge);
    }
}
void * papir_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&papir);
        sem_wait(&papir_penge);
    }
}
void * lighter_pusher(void *arg){
    for(int i=0;i<2000;i++){
        sem_post(&lighter);
        sem_wait(&lighter_penge);
    }
}

void * ryger_med_tobak(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&papir);
        sem_wait(&lighter);
        printf("%ld ryger cigaret %d \n", (long)arg, i);
        sem_post(&papir_penge);
        sem_post(&lighter_penge);
    }
    printf("%ld ryger død\n", (long)arg);
}
void * ryger_med_papir(void *arg){
    for(int i=0;i<1000;i++){
        sem_wait(&lighter);
        sem_wait(&tobak);
        printf("%ld ryger cigaret %d\n", (long)arg,i);
        [sem post(&tobak penge);]
```



```
    sem_post(&lighter_penge);
}
printf("%ld ryger død\n", (long)arg);
}

void * ryger_med_lighter(void *arg){
    for(int i=0;i<1000;i++){
        [sem_wait(&papir);]
        [sem_wait(&tobak);]
        printf("%ld ryger cigaret %d \n", (long)arg,i);
        sem_post(&papir_penge);
        [sem_post(&tobak_penge);]
    }
    printf("%ld ryger død\n", (long)arg);
}

int main(int argc,char argv[]){
    sem_init(&tobak, 0, 0);
    sem_init(&papir, 0, 0);
    sem_init(&lighter, 0, 0);
    sem_init(&tobak_penge, 0, 0);
    sem_init(&papir_penge, 0, 0);
    sem_init(&lighter_penge, 0, 0);

    pthread_t tp, lp,pp, rl,rt,rp;
    Pthread_create(&tp,NULL,tobak_pusher,(void*)0);
    Pthread_create(&lp,NULL,lighter_pusher,(void*)1);
    Pthread_create(&pp,NULL,papir_pusher,(void*)2);
    Pthread_create(&rl,NULL,ryger_med_lighter,(void*)3);
    Pthread_create(&rt,NULL,ryger_med_tobak,(void*)4);
    Pthread_create(&rp,NULL,ryger_med_papir,(void*)5);

    Pthread_join(tp,NULL); Pthread_join(lp,NULL);
    Pthread_join(pp,NULL); Pthread_join(rl,NULL);
    Pthread_join(rt,NULL); Pthread_join(rp,NULL);
    printf("All Dead\n");
    return 0;
}
```

[Finish review](#)



Status Finished	Started Saturday, 7 June 2025, 6:11 PM	Completed Saturday, 7 June 2025, 6:11 PM
Duration 33 secs	Marks 0.00/3.00	Grade 0.00 out of 10.00 (0%)

Question 1

[Flag question](#)

Not answered

Marked out of 1.00

Del 1/3. Opstil de nødvendige erklæringer og initialisering af mutex og condition variable, samt eventuelle globale delte variable.

Listing 7: Erklæringer.

```
1 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
2 pthread_cond_t waiters = PTHREAD_COND_INITIALIZER;  
3 int noArrived=0;
```

Question 2

[Flag question](#)

Not answered

Marked out of 1.00

Del 2/3. Opstil koden til synkronisering, som kan indsættes omkring linie 22.

Listing 8: Synkroniseringskode.

```
1
2 pthread_mutex_lock(&mutex);
3 noArrived++;
4
5 if(noArrived==N) {
6     pthread_cond_broadcast(&waiters);
7 } else {
8     while(noArrived<N) {
9         pthread_cond_wait(&waiters,&mutex);
10    }
11 }
12 pthread_mutex_unlock(&mutex);
```

Question 3

[Flag question](#)

Not answered

Marked out of 1.00

Del 3/3. Beskriv kort virkemåden af din løsning, eller løsningsidé.



Ideen er at lade en global variable optælle antallet af ankomne tråde. Sålænge antallet ikke har nået 3, skal trådene blokeres vha. `cond_wait`. Den sidst ankomne vækker alle øvrige vha `cond_broadcast`. Optælling af antallet af ankomne tråde skal ske udeleligt, og derfor beskyttes med en lås. Endvidere skal brug af wait og (bør) signal/broadcast på en condition variable ske mens låsen holdes. Vi husker at vække ventende så der ikke bliver baglås. Kaldet til broadcast sker kun når antallet af fremmødte er N, så unødvendig vækning/kald til `condbroadcast` undgås.

[Finish review](#)